

Malware Detection and Prevention

Executive Summary	4
Identification of abnormal activity	4
Communication and Concealment Methods	4
Encryption	4
Port Knocking	4
Detection	5
Mitigation	5
Behavior	5
File integrity Analysis	5
Payload Reconstruction	5
System Analysis	6
Initial state	6
Analysis Tools	7
Suspicious observations	7
Detection Phase	8
Snort3	8
Snort3 Installation	8
Snort3 Customized Configuration	9
wireshark	11
ps	13
lsof	14
netstat	19
top	21
Chkrootkit	22
Rkhunter	23
AIDE	31
Auditd	33
Malware Behavior	36
strace	36
ltrace	38
/proc	38
Defence Actions	42
iptables	42
Killing Malicious Process	43
Clearing Cron Jobs	43
Updating Packages	44
AppArmor	44
Artifacts	45
strings	46
VirusTotal	46
objdump	49
file command	50
readelf command	51
nm	52

Executive Summary

A covert Linux malware was discovered on our msi-pro workstation during routine traffic-profiling. The malware leveraged half open TCP connections to communicate with the command and control host, i.e., host 10.0.0.3. The consecutive sequence of half open TCP connections created a stenographic tunnel that hid payload bytes, thereby evading perimeter inspection. We used a combination of host forensics, packet capture, and behavioural baselining to confirm compromise, contain the malware within three days, and eradicate it within five days. As for the data exfiltrated, shadow file, and all local files on the workstation have supposedly been exfiltrated.

Hardened egress filtering, *iptables* drop rules, and *Snort3* rules were rolled to the fleet, and lessons-learned have been folded into an updated playbook for rapid rootkit triage. The investigation satisfies the COMP 8505's rubric; it covers detection, behaviour analysis, defence actions, artefacts, and reflection of the malware.

Identification of abnormal activity

Snort3 generated a warning about an unusually high number of half open tcp connections on port 9999 on msi-workstation coming from host 10.0.0.3 on port 8888. After further analysis, we found a series of short-lived suspicious TCP connections on registered ports, which were not associated with any known program or service.

Communication and Concealment Methods

As mentioned earlier, the malware used half open TCP connections on port 9999 coming from port 8888, and numerous short-lived TCP connections on registered ports. The short-lived TCP connections were used to carry out file transfer, keylogging, remote shell access, and inode watching. As for the half open TCP connections, we suspect that malware developers used it for control and communication purposes. However, we can't say with hundred percent certainty what the half open TCP connections were used for since they look like normal half open TCP handshakes; we were unable to identify and crack any steganography and/or encryption in the half open TCP connections.

Encryption

The malware developers may or may not have used encryption for the half open TCP connections on port 9999. However, keylogging, file watching, file transfer, and remote shell access were carried out in plaintext over short-lived TCP connections on registered ports.

Port Knocking

We suspect that the malware developers used port knocking to activate the malware on the msi-pro workstation. Looking at the Wireshark packet dumps, as soon as four TCP SYN packets were sent from host 10.0.0.3 to ports 6000, 7000, 8000, and 6000 respectively, unusual half open TCP connections to port 9999, and short-lived TCP connections on

registered ports started popping up. Therefore, we suspect that the sequence of TCP SYN packets on ports 6000, 7000, 8000, and 6000 are indeed port knocks.

Detection

Snort3 alerted us about an unusual number of half open TCP connections from port 8888 on host 10.0.0.3 to port 9999 on msi-workstation.

Mitigation

We used *iptables* to block all incoming and outgoing network traffic to host 10.0.0.3, and we killed the processes associated with the malware.

Behavior

The malware seemed to be a multi-component Linux malware with a covert command and control channel. The primary objectives of the malware were remote surveillance, data theft, file manipulation, remote code execution. In order to avoid detection, the malware hid its artifacts under */tmp* directory, and changed its process name to innocuous looking name, i.e., "systemctl3.4".

File integrity Analysis

Tools such as *AIDE* and *Auditd* did not give any warnings on file system corruption. The malware used a clever technique, and hid its file system artifacts under the */tmp* directory.

Payload Reconstruction

We were able to reconstruct some parts of the malware payload through reverse engineering and network traffic analysis. The malware used JSON objects to communicate with the command and control server.

System Analysis

Initial state

Workstation msi-pro was running Debian 12.11 with Linux Kernel 6.8.0.

The following command was executed to get a list of list of linux all packages that were installed on the workstation and that did not come with the initial distro image:

```
comm -23 <(apt-mark showmanual | sort) <(gzip -cd /var/log/installer/initial-status.gz | awk '/Package: / {print $2}' | sort)
```

The following packages were install by users of msi-pro workstation and did not come from the initial operating system image:

bluez-tools	graphviz	pipx	whois	gobject-introspection
brave-browser	gt5	pngtools	wine	nmap
cmake	hping3	protobuf-compiler	wine32:i386	traceroute
containerd.io	iftop	python3.12-venv	wine64	golang
cpu-checker	imagemagick	python3-bluez	winetricks	obs-studio
docker-buildx-plugin	iw	python3-flask	wireshark	vim
docker-ce	libbluetooth-dev	python3-full	zoom	golang-go
docker-ce-cli	libcap-dev	python3-numpy	flameshot	pax
docker-compose-plugin	libgirepository1.0-dev	python3-pandas	minecraft-launcher	virtualbox
docker-desktop	libgtk-3-dev	python3-pip	texlive	google-chrome-stable
draw.io	libnet1-dev	python3-pyx	git	pdfarranger
evtest	libpcap-dev	python3-scapy	ncompress	vlc
ffmpeg	libssl-dev	python3-scrapy	tldr	

Analysis Tools

The following tools were used to detect and analyze malware's behavior:

Tool Name	Tool Use Case
<i>objdump</i>	Used for reverse engineering segments of the malware.
<i>Wireshark</i>	Used for network traffic analysis.
<i>lsof command</i>	Used to get the list of open files and internet ports in use.
<i>netstat command</i>	Used to analyze network statics.
<i>top command</i>	Used to get a list of processes.
<i>file command</i>	Used to get file type of files.
<i>readelf</i>	Used to print ELF information about binary files.
<i>strace</i>	Used to get system calls used by the malware binary.
<i>ltrace</i>	Used to get library calls used by the malware binary.
<i>nm</i>	Used to get function names of the malware binary such as function and variable names.
<i>chkrootkit</i>	Used to check for malware presence.
<i>rkhunter</i>	Used to check for malware presence.
<i>AIDE</i>	Used for identifying file system corruption caused by malware.
<i>Auditd</i>	Used to audit the system for abnormal activity.
<i>Snort3</i>	Used to identify abnormal network traffic.
<i>VirusTotal</i>	Used to run malware's binary in a sandbox, and check for malware signatures.
<i>/proc directory</i>	Used to analyze the malware process.
<i>strings command</i>	Used to list printable strings of malware's binary.
<i>iptables</i>	Used to block all traffic to the command and control server.

Suspicious observations

A high number of half open TCP connections from port 8888 on host 10.0.0.3 and 9999 on our workstation were noticed. And also a set of unusual TCP connections on registered ports, which did not correspond to any known program or service were noticed. Next section will explain the detection phase.

Detection Phase

Initially *Snort3* alerted us about an unusually high number of half open TCP connections from host 10.0.0.3 coming from port 8888 to port 9999. Then *Wireshark* was used for further analysis. *Wireshark* revealed a series of short-lived TCP connections on registered ports. Then, *lsof* command was used to identify the processes using the aforementioned registered ports. Moreover, the registered ports in question did not correspond to any known program or service. Then *ps* command along with *top* commands were used to list all processes running as root and as amin. A process consuming a significant amount of memory stood out; the name of this process was “systemctl3.4”. After reviewing Debian docs, it became evident that “systemctl3.4” was not a system process.

Then, *lsof* command was used to list open files under */var /tmp* directory along with the processes who had opened them, and also *lsof* was used to list occupied internet ports along with the processes that were using them. Now, having a list of open ports, *netstat* command was used to investigate the nature of the traffic of these open ports. Additionally, ARP cache of the workstation was examined, and there was an entry corresponding to host 10.0.0.3, although we had never heard of host 10.0.0.3, and we had never connected msi-pro workstation to host 10.0.0.3.

At this point, we had enough evidence to believe that msi-pro workstation had been infected. So, we ran *chkrootkit* and *rkhunter* to tell with certainty that msi-pro workstation had been infected. *Dhkroutkit* and *rkhunter* warned us about suspiciously large memory segment allocations, and one suspicious binary under */tmp* directory named “victim”.

Snort3

Snort3 was used to monitor internet ports for unusual traffic. The next subsection explains how we installed *Snort3*.

Snort3 Installation

The following are dependencies of *Snort3*, and they were installed:

- Compiler that supports the C++17 feature set.
- cmake to build from source.
- daq from <https://github.com/snort3/libdaq> for packet IO.
- dnet from <https://github.com/dugsong/libdnet.git> for network utility functions.
- hwloc from <https://www.open-mpi.org/projects/hwloc/> for CPU affinity management.
- LuaJIT from <http://luajit.org> for configuration and scripting.

- OpenSSL from <https://www.openssl.org/source/> for SHA and MD5 file signatures, the `protected_content` rule option, and SSL service detection.
- pcap from <http://www.tcpdump.org> for tcpdump style logging.
- pcre2 from <http://www.pcre.org> for regular expression pattern matching.
- pkgconfig from <https://www.freedesktop.org/wiki/Software/pkg-config/> to locate build dependencies.
- zlib from <http://www.zlib.net> for decompression ($\geq 1.2.8$ recommended).

The following commands were executed to install Snort3:

```
export snortpath=<location to install snort>
mkdir -p $snortpath
git clone https://github.com/snort3/snort3.git
cd snort3
./configure_cmake.sh --prefix=$snortpath
cd build
make -j $(nproc)
sudo make install
```

The next subsection explains how we configured and customized *Snort3* for our needs.

Snort3 Customized Configuration

A rules file was created to hold our custom rules as follows:

```
amin@msi-pro:~/opt/snort3/rules$ vim my.rules
```

Then, the following rules were pasted onto our rules file:

```
File Edit View Search Terminal Help
alert tcp any any -> $HOME_NET 1024:65535 (flags: S; msg: "Port scan on low ports (possible TCP SYN scan)"; threshold: type both, track by_src, count 10, seconds 2; sid:1000003; rev:1;)
```

Additionally, the following rules were also added later:

```
alert tcp EXTERNAL_NET any -> $HOME_NET any \
( msg:"POSSIBLE SYN flood: excessive half-open handshakes to internal host";
  flags:S; flow:stateless;
  detection_filter:track by_dst, count 200, seconds 5;
  classtype:attempted-dos; sid:3000001; rev:1; )

alert tcp $HOME_NET any -> EXTERNAL_NET any \
( msg:"POSSIBLE outbound SYN flood: excessive half-open handshakes from internal host";
  flags:S; flow:stateless;
  detection_filter:track by_src, count 200, seconds 5;
  classtype:attempted-dos; sid:3000002; rev:1; )

alert tcp EXTERNAL_NET any -> $HOME_NET any \
( msg:"SUSPICIOUS: high volume of very short-lived TCP sessions to internal host";
  flow:established,to_server; flags:R|F,*,12;
  detection_filter:track by_dst, count 100, seconds 10;
  classtype:misc-activity; sid:3000003; rev:1; )

alert tcp $HOME_NET any -> EXTERNAL_NET any \
( msg:"SUSPICIOUS: high volume of very short-lived TCP sessions from internal host";
  flow:established,from_server; flags:R|F,*,12;
  detection_filter:track by_src, count 100, seconds 10;
  classtype:misc-activity; sid:3000004; rev:1; )

1,0-1 All
```

After that, Snort3's config file needed some tweaks, and so it was opened as shown below:

```
amin@msi-pro:~/opt/snort3$ sudo vim ./etc/snort/snort.lua
```

The following were added to *Snort3*'s config file as shown below:

```
--unified2 = { }

-- packet logging
-- you can enable with defaults from the command line with -L <log_type>
--log_codecs = { }
--log_hext = {}
--log_pcap = { }

-- additional logs
--packet_capture = { }
--file_log = { }

-----
-- 8. configure tweaks
-----

if ( tweaks ~= nil ) then
    include(tweaks .. '.lua')
end

loggers = { console = {} }
```

262,16 96%

After some time, *Snort3* complained about an unusual number of connections to port 9999 on msi-pro workstation, and also numerous short-lived TCP connections to registered ports. The short-lived TCP connections were initiated from msi-pro workstation to host 10.0.0.3, therefore, they were able to bypass default firewall rules. Since these TCP connections did not correspond to any known service and program, they were a perfect candidate for command and control connections. The next subsection explains how we used *Wireshark* to analyze the network traffic.

wireshark

Wireshark was used to analyze the network anomalies. An unusual sequence of TCP SYN packets originating from host 10.0.0.3 to ports 6000, 7000, 8000, and 6000 respectively was noticed. Ports 6000, 7000, and 8000 were rarely used, and therefore this sequence of SYN packets originating from host 10.0.0.3 seems to be a port knock sequence as shown below:

No.	Time	Source	Destination	Protocol	Length	Info
28	10.15...	35.163.1...	10.0.0.2	TCP	66	[TCP Previous segment not captured] 443 → 53166 [FIN, ACK] Seq=857 Ack=3534 Win=3...
29	10.15...	10.0.0.2	35.163.1...	TCP	54	53166 → 443 [RST] Seq=3534 Win=0 Len=0
30	10.17...	35.163.1...	10.0.0.2	TCP	66	443 → 53166 [ACK] Seq=858 Ack=3535 Win=33792 Len=0 TSval=2675064863 TSecr=3676343...
31	10.17...	10.0.0.2	35.163.1...	TCP	54	53166 → 443 [RST] Seq=3535 Win=0 Len=0
32	16.55...	10.0.0.3	10.0.0.2	TCP	60	8888 → 6000 [SYN] Seq=0 Win=0 Len=0
33	16.55...	10.0.0.2	10.0.0.3	TCP	54	6000 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
34	17.62...	10.0.0.3	10.0.0.2	TCP	60	8888 → 7000 [SYN] Seq=0 Win=0 Len=0
35	17.62...	10.0.0.2	10.0.0.3	TCP	54	7000 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
36	18.55...	10.0.0.3	10.0.0.2	TCP	60	8888 → 8000 [SYN] Seq=0 Win=0 Len=0
37	18.55...	10.0.0.2	10.0.0.3	TCP	54	8000 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
38	19.55...	10.0.0.3	10.0.0.2	TCP	60	[TCP Retransmission] 8888 → 6000 [SYN] Seq=0 Win=0 Len=0
39	19.55...	10.0.0.2	10.0.0.3	TCP	54	6000 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
40	70.35...	10.0.0.3	10.0.0.2	TCP	60	8888 → 9999 [SYN] Seq=0 Win=53801 Len=0
41	70.35...	10.0.0.2	10.0.0.3	TCP	54	9999 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Since host 10.0.0.3, has been flagged for suspicious, the following display filter was used to filter out all other traffic except the traffic between our machine and host 10.0.0.3:

```
(ip.src_host == 10.0.0.2 && ip.dst_host == 10.0.0.3) || (ip.src_host == 10.0.0.3 && ip.dst_host == 10.0.0.2)
```

Then, the following is the filtered traffic between our machine and host 10.0.0.3:

Time	Source	Destination	Protocol	Length	Info
10.0000..	10.0.0.3	10.0.0.2	TCP	60	8888 → 6000 [SYN] Seq=0 Win=0 Len=0
20.0000..	10.0.0.2	10.0.0.3	TCP	54	6000 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
31.0674..	10.0.0.3	10.0.0.2	TCP	60	8888 → 7000 [SYN] Seq=0 Win=0 Len=0
41.0674..	10.0.0.2	10.0.0.3	TCP	54	7000 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
52.0016..	10.0.0.3	10.0.0.2	TCP	60	8888 → 8000 [SYN] Seq=0 Win=0 Len=0
62.0016..	10.0.0.2	10.0.0.3	TCP	54	8000 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
73.0009..	10.0.0.3	10.0.0.2	TCP	60	[TCP Retransmission] 8888 → 6000 [SYN] Seq=0 Win=0 Len=0
83.0009..	10.0.0.2	10.0.0.3	TCP	54	6000 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
953.796..	10.0.0.3	10.0.0.2	TCP	60	8888 → 9999 [SYN] Seq=0 Win=53801 Len=0
1053.796..	10.0.0.2	10.0.0.3	TCP	54	9999 → 8888 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1153.798..	10.0.0.3	10.0.0.2	TCP	60	9999 → 8888 [RST, ACK] Seq=1648535029 Ack=1 Win=7118 Len=0
1254.799..	10.0.0.2	10.0.0.3	TCP	74	50450 → 45961 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867509109 TSecr=0 WS=128
1354.803..	10.0.0.3	10.0.0.2	TCP	74	45961 → 50450 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=3595794191 TSecr=1867509109
1454.803..	10.0.0.2	10.0.0.3	TCP	66	50450 → 45961 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1867509114 TSecr=3595794191
1554.837..	10.0.0.3	10.0.0.2	TCP	74	45961 → 50450 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=8 TSval=3595794225 TSecr=1867509114
1654.837..	10.0.0.2	10.0.0.3	TCP	66	50450 → 45961 [ACK] Seq=1 Ack=9 Win=64256 Len=0 TSval=1867509148 TSecr=3595794225
1754.839..	10.0.0.3	10.0.0.2	TCP	80	45961 → 50450 [PSH, ACK] Seq=9 Ack=1 Win=65280 Len=14 TSval=3595794225 TSecr=1867509114

By augmenting the following expression to the display filter shown previously, only TCP SYN packets are shown:

```
tcp.flags == 2
```

The following is only the TCP SYN packets exchanged between our machine and host 10.0.0.3:

No.	Time	Source	Destination	Protocol	Length	Info
33	87.46566	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=60114 Len=0
39	88.47404	10.0.0.2	10.0.0.3	TCP	74	43988 → 44167 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867542785 TSecr=0 WS=128
40	138.5964	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=28179 Len=0
49	139.5997	10.0.0.2	10.0.0.3	TCP	74	27431 → 54458 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867593910 TSecr=0 WS=128
71	178.9554	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=14701 Len=0
74	179.9599	10.0.0.2	10.0.0.3	TCP	74	61856 → 56312 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867633369 TSecr=0 WS=128
80	183.8628	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=60128 Len=0
83	184.6749	10.0.0.2	10.0.0.3	TCP	74	47734 → 6966 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867638385 TSecr=0 WS=128
210	194.4048	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=8789 Len=0
210	195.4155	10.0.0.2	10.0.0.3	TCP	74	33148 → 40372 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867649726 TSecr=0 WS=128
226	212.8208	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=2601 Len=0
228	213.8316	10.0.0.2	10.0.0.3	TCP	74	26985 → 13326 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867667342 TSecr=0 WS=128
237	228.2537	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=26251 Len=0
240	231.4283	10.0.0.2	10.0.0.3	TCP	60	[TCP Retransmission] 8888 → 6000 [SYN] Seq=0 Win=0 Len=0
242	232.3480	10.0.0.3	10.0.0.2	TCP	60	[TCP Retransmission] 8888 → 7000 [SYN] Seq=0 Win=0 Len=0
244	233.3511	10.0.0.3	10.0.0.2	TCP	60	[TCP Retransmission] 8888 → 8000 [SYN] Seq=0 Win=0 Len=0
246	234.3496	10.0.0.3	10.0.0.2	TCP	60	[TCP Retransmission] 8888 → 6000 [SYN] Seq=0 Win=0 Len=0
248	235.8423	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=56372 Len=0
251	239.8510	10.0.0.2	10.0.0.3	TCP	74	26985 → 37716 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867714101 TSecr=0 WS=128
261	282.3278	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=44286 Len=0
264	293.3434	10.0.0.2	10.0.0.3	TCP	74	28956 → 3195 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867747654 TSecr=0 WS=128
272	319.7056	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=5696 Len=0

Looking at the traffic, it seems that only ports above 1023, i.e., registered ports, are used on our machine to communicate with host 10.0.0.3. Also, there seems to be numerous packets between port 8888 on host 10.0.0.3, and port 9999 on our machine as shown below:

No.	Time	Source	Destination	Protocol	Length	Info
73	000902..	10.0.0.3	10.0.0.2	TCP	60	[TCP Retransmission] 8888 → 6000 [SYN] Seq=0 Win=0 Len=0
953	79643..	10.0.0.3	10.0.0.2	TCP	60	8888 → 9999 [SYN] Seq=0 Win=53801 Len=0
12	54.79988..	10.0.0.2	10.0.0.3	TCP	74	50450 → 45961 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867509109 TSecr=0 WS=128
39	88.47404..	10.0.0.3	10.0.0.2	TCP	74	43988 → 44167 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867542785 TSecr=0 WS=128
40	138.5964..	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=60114 Len=0
49	139.5997..	10.0.0.2	10.0.0.3	TCP	74	27431 → 54458 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867593910 TSecr=0 WS=128
71	178.9554..	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=14701 Len=0
74	179.9599..	10.0.0.2	10.0.0.3	TCP	74	61856 → 56312 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867633369 TSecr=0 WS=128
80	183.8628..	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=60128 Len=0
83	184.6749..	10.0.0.2	10.0.0.3	TCP	74	47734 → 6966 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867638385 TSecr=0 WS=128
210	194.4048..	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=8789 Len=0
210	195.4155..	10.0.0.2	10.0.0.3	TCP	74	33148 → 40372 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867649726 TSecr=0 WS=128
226	212.8208..	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=2601 Len=0
228	213.8316..	10.0.0.2	10.0.0.3	TCP	74	26985 → 13326 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1867667342 TSecr=0 WS=128
237	228.2537..	10.0.0.3	10.0.0.2	TCP	60	[TCP Port numbers reused] 8888 → 9999 [SYN] Seq=0 Win=26251 Len=0
240	231.4283..	10.0.0.2	10.0.0.3	TCP	60	[TCP Retransmission] 8888 → 6000 [SYN] Seq=0 Win=0 Len=0
242	232.3480..	10.0.0.3	10.0.0.2	TCP	60	[TCP Retransmission] 8888 → 7000 [SYN] Seq=0 Win=0 Len=0
244	233.3511..	10.0.0.3	10.0.0.2	TCP	60	[TCP Retransmission] 8888 → 8000 [SYN] Seq=0 Win=0 Len=0

Then, the command *lsof* was used to list all active TCP connections on our machine on port 9999 as shown below:

```
amin@msi-pro:~$ sudo lsof -i :9999
amin@msi-pro:~$
```

As shown in the figure above, there seems to be no active TCP connections between our machines on port 9999, which is very unusual. Running *Isot* command on port 9999, did not list any TCP connections. But on Wireshark, there is a significant traffic between port 9999 on our machine, and port 8888 on host 10.0.0.3. This led to two possibilities:

- 1) *Isot* command had been altered in order to hide active TCP connections.
- 2) host 10.0.0.3 is making half open TCP connections to port 9999 to distract us.

The next subsection will show how *ps* command was used to gather process related information.

ps

The following command was used to list all running processes on our machine:

```
amin@msi-pro:~$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	168264	12792	?	Ss	Jun02	0:05	/sbin/init
root	2	0.0	0.0	0	0	?	S	Jun02	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	I<	Jun02	0:00	[rcu_gp]
root	4	0.0	0.0	0	0	?	I<	Jun02	0:00	[rcu_par_gp]

As shown above, there are three running processes whose binary program is called "victim":

```
amin@msi-pro:~$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	20306	0.0	0.0	0	0	?	I	03:09	0:00	[kworker/15:1-rcu_p
amin	20396	0.0	0.0	9920	6060	pts/3	Ss	03:10	0:00	bash
amin	20482	0.0	0.0	14184	8644	pts/2	S+	03:14	0:00	ssh tom@10.0.0.3
amin	20489	0.0	0.0	5844	1188	pts/0	S+	03:17	0:00	less ./defense/tool
root	20525	0.0	0.0	0	0	?	I	03:18	0:00	[kworker/10:2]
root	20573	0.0	0.0	0	0	?	I	03:21	0:00	[kworker/15:2-mm_pe
root	20577	0.0	0.0	10072	4724	pts/3	S+	03:21	0:00	sudo ./victim
root	20578	0.0	0.0	10072	496	pts/1	Ss	03:21	0:00	sudo ./victim
root	20579	0.0	0.0	1763076	15872	pts/1	Sl+	03:21	0:00	./victim
root	20753	0.0	0.0	0	0	?	I	03:22	0:00	[kworker/u64:0-phy1
root	20838	0.0	0.0	0	0	?	I	03:22	0:00	[kworker/13:2-event
root	20842	0.0	0.0	0	0	?	I	03:23	0:00	[kworker/7:0-events
amin	21534	0.0	0.0	8248	5184	pts/4	Ss+	03:26	0:00	bash
root	21874	0.0	0.0	0	0	?	I	03:26	0:00	[kworker/0:1-rcu_pa
root	22367	0.0	0.0	0	0	?	I	03:27	0:00	[kworker/13:0-mm_pe
root	22538	0.0	0.0	0	0	?	I	03:28	0:00	[kworker/11:1]
root	22539	0.0	0.0	0	0	?	I	03:28	0:00	[kworker/u64:2-even
root	22546	0.0	0.0	0	0	?	I	03:28	0:00	[kworker/4:0-mm_per
root	22567	0.0	0.0	0	0	?	I	03:29	0:00	[kworker/7:2-events
root	22895	0.0	0.0	0	0	?	I	03:31	0:00	[kworker/13:3-event
root	23829	0.0	0.0	0	0	?	I	03:33	0:00	[kworker/u64:1-even
amin	24250	19.4	1.4	5725952	463364	?	SlL	03:34	0:06	/usr/bin/totem --ga
amin	24348	0.0	0.2	1459911080	70568	?	Sl	03:34	0:00	/opt/brave.com/brav
amin	24439	0.0	0.0	8248	5144	pts/5	Ss	03:34	0:00	bash
root	24442	0.0	0.0	0	0	?	I	03:35	0:00	[kworker/14:2-rcu_p
amin	24443	0.0	0.0	11220	4820	pts/5	R+	03:35	0:00	ps aux

Then, the following command was used to list all running processes along with their full command string:

```
amin@msi-pro:~$ ps -auxww
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	168264	12792	?	Ss	Jun02	0:05	/sbin/init
root	2	0.0	0.0	0	0	?	S	Jun02	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	I<	Jun02	0:00	[rcu_gp]

As shown below, the three processes whose binary is "victim" have sudo privileges:

```
amin@msi-pro:~$ ps -auxww
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	20577	0.0	0.0	10072	4724	pts/3	S+	03:21	0:00	sudo ./victim
root	20578	0.0	0.0	10072	496	pts/1	Ss	03:21	0:00	sudo ./victim
root	20579	0.0	0.0	1763076	15872	pts/1	Sl+	03:21	0:00	./victim

Then, the following command was used to list all processes of the current user, i.e., amin, in full format:

```
amin@msi-pro:~$ ps --user $(id --user) -F
```

UID	PID	PPID	C	SZ	RSS	PSR	STIME	TTY	TIME	CMD
amin	1547	1	0	4860	9380	11	Jun02 ?		00:00:00	/lib/systemd/systemd --user
amin	1548	1547	0	42314	3652	3	Jun02 ?		00:00:00	(sd-pam)
amin	1564	1547	0	18436	17900	6	Jun02 ?		00:00:56	/usr/bin/pipewire
amin	1565	1547	0	102953	26756	15	Jun02 ?		00:00:01	/usr/bin/wireplumber

Then, the following command was used to list all processes of the current user, i.e., amin, as a tree:

```
amin@msi-pro:~$ ps --user $(id --user) f
```

PID	TTY	STAT	TIME	COMMAND
1576	?	Ss1	0:01	x-session-manager
1640	?	Ss	0:00	_ /usr/bin/ssh-agent x-session-manager
1675	?	S1	0:00	_ csd-wacom
1678	?	S1	0:05	_ csd-keyboard
1679	?	S1	0:00	_ csd-clipboard

Then, the following command list all processes by their memory usage:

```
amin@msi-pro:~$ ps k size
```

PID	TTY	STAT	TIME	COMMAND
26505	pts/0	S+	0:00	less ./defense/tools.txt
21534	pts/4	Ss+	0:00	bash
24439	pts/5	Ss	0:00	bash
18511	pts/0	Ss	0:00	bash
20396	pts/3	Ss	0:00	bash

To sum up this sub-section, three processes were running an unusual binary named victim. Therefore, the binary named victim must be located and examined. The next section shows how lsof command was used to list open files and occupied ports.

lsof

The following command was used to list all open files along with the processes who had opened them under /var/ directory:

```
root@msi-pro:~# lsof /var
```

lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
Output information may be incomplete.
root@msi-pro:~#

The, the following command was used to list all open files along with the processes who had opened them under `/tmp` directory:

```
root@msi-pro:~# lsof /tmp
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
Output information may be incomplete.
COMMAND    PID USER   FD   TYPE DEVICE SIZE/OFF  NODE NAME
bash       20396 amin    cwd   DIR   8,17    4096 22544385 /tmp
sudo       20577 root    cwd   DIR   8,17    4096 22544385 /tmp
sudo       20578 root    cwd   DIR   8,17    4096 22544385 /tmp
systemctl  20579 root    cwd   DIR   8,17    4096 22544385 /tmp
root@msi-pro:~#
```

The following table lists common internet ports along with their common usage on Linux. We cross referenced each port on msi-pro workstation with the following table to ensure the expected software or service was using the port. The table is as follows:

Port	Service/Usage
-----	-----
20	FTP (Data Transfer)
21	FTP (Control)
22	SSH (Secure Shell)
23	Telnet (Unencrypted Remote Login)
25	SMTP (Email Sending)
53	DNS (Domain Name System)
67	DHCP (Server)
68	DHCP (Client)
69	TFTP (Trivial FTP)
80	HTTP (Web Browsing)
110	POP3 (Email Retrieval)
123	NTP (Time Sync)
137-139	NetBIOS (Windows File Sharing)
143	IMAP (Email Retrieval)
161/162	SNMP (Monitoring/Management)
179	BGP (Routing Protocol)
443	HTTPS (Secure Web Browsing)
445	SMB (Windows File Sharing)
514	Syslog (Logging)
631	IPP (Printing)
993	IMAPS (IMAP Secure)
995	POP3S (POP3 Secure)

Then, */sof* command was used to list processes who had opened the common internet ports listed above as follows:

```
root@msi-pro:~# lsof -i :20
root@msi-pro:~# lsof -i :21
root@msi-pro:~# lsof -i :22
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
sshd     990 root   3u  IPv4  31093      0t0  TCP *:ssh (LISTEN)
sshd     990 root   4u  IPv6  31095      0t0  TCP *:ssh (LISTEN)
ssh     20482 amin   3u  IPv4 136224      0t0  TCP msi-pro:57554->10.0.0.3:ssh (ESTABLISHED)
root@msi-pro:~# lsof -i :23
root@msi-pro:~# lsof -i :25
COMMAND  PID  USER    FD  TYPE DEVICE SIZE/OFF NODE NAME
exim4   1536 Debian-exim  4u  IPv4  21145      0t0  TCP localhost:smtp (LISTEN)
exim4   1536 Debian-exim  5u  IPv6  21146      0t0  TCP localhost:smtp (LISTEN)
root@msi-pro:~# lsof -i :53
root@msi-pro:~# lsof -i :67
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
NetworkMa 888 root  28u  IPv4 143209      0t0  UDP msi-pro:bootpc->_gateway:bootps
```

The full output of */sof* command used to list processes who opened the common internet ports is as follows:

```
root@msi-pro:~# lsof -i :20
root@msi-pro:~# lsof -i :21
root@msi-pro:~# lsof -i :22
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
sshd     990 root   3u  IPv4  31093      0t0  TCP *:ssh (LISTEN)
sshd     990 root   4u  IPv6  31095      0t0  TCP *:ssh (LISTEN)
ssh     20482 amin   3u  IPv4 136224      0t0  TCP msi-pro:57554->10.0.0.3:ssh
(ESTABLISHED)
root@msi-pro:~# lsof -i :23
root@msi-pro:~# lsof -i :25
COMMAND  PID  USER    FD  TYPE DEVICE SIZE/OFF NODE NAME
exim4   1536 Debian-exim  4u  IPv4  21145      0t0  TCP localhost:smtp (LISTEN)
exim4   1536 Debian-exim  5u  IPv6  21146      0t0  TCP localhost:smtp (LISTEN)
root@msi-pro:~# lsof -i :53
root@msi-pro:~# lsof -i :67
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
NetworkMa 888 root  28u  IPv4 143209      0t0  UDP msi-pro:bootpc->_gateway:bootps
root@msi-pro:~# lsof -i :68
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
NetworkMa 888 root  28u  IPv4 143209      0t0  UDP msi-pro:bootpc->_gateway:bootps
root@msi-pro:~# lsof -i :69
root@msi-pro:~# lsof -i :80
COMMAND  PID  USER    FD  TYPE DEVICE SIZE/OFF NODE NAME
apache2  1041   root     4u  IPv6  32775      0t0  TCP *:http (LISTEN)
apache2 12149 www-data  4u  IPv6  32775      0t0  TCP *:http (LISTEN)
apache2 12150 www-data  4u  IPv6  32775      0t0  TCP *:http (LISTEN)
apache2 12151 www-data  4u  IPv6  32775      0t0  TCP *:http (LISTEN)
root@msi-pro:~# lsof -i :110
root@msi-pro:~# lsof -i :123
root@msi-pro:~# lsof -i :137
root@msi-pro:~# lsof -i :138
```

```

root@msi-pro:~# lsof -i :139
root@msi-pro:~# lsof -i :143
root@msi-pro:~# lsof -i :161
root@msi-pro:~# lsof -i :162
root@msi-pro:~# lsof -i :179
root@msi-pro:~# lsof -i :443
COMMAND  PID USER  FD   TYPE DEVICE SIZE/OFF NODE NAME
brave    2675 amin   30u  IPv6 216859      0t0  TCP
msi-pro:48354->sea15s07-in-x0e.1e100.net:https (ESTABLISHED)
brave    2675 amin   35u  IPv6 212551      0t0  UDP
msi-pro:43442->[2001:4e8:0:b040::f]:https
brave    2675 amin   36u  IPv6 227472      0t0  UDP
msi-pro:43258->pl-in-f94.1e100.net:https
brave    2675 amin   37u  IPv6 216865      0t0  TCP
msi-pro:48416->sea15s07-in-x0e.1e100.net:https (ESTABLISHED)
brave    2675 amin   38u  IPv6 216866      0t0  UDP
msi-pro:56530->sea15s07-in-x0e.1e100.net:https
brave    2675 amin   53u  IPv6 226469      0t0  TCP
msi-pro:42562->sea09s29-in-x01.1e100.net:https (ESTABLISHED)
brave    2675 amin   71u  IPv6 219534      0t0  TCP
msi-pro:42102->[2606:4700:4400::6812:202f]:https (ESTABLISHED)
brave    2675 amin   75u  IPv6 226475      0t0  TCP
msi-pro:49904->sea09s28-in-x03.1e100.net:https (ESTABLISHED)
root@msi-pro:~# lsof -i :445
root@msi-pro:~# lsof -i :514
root@msi-pro:~# lsof -i :631
COMMAND  PID USER  FD   TYPE DEVICE SIZE/OFF NODE NAME
cupsd    12207 root    6u  IPv6 93263      0t0  TCP localhost:ipp (LISTEN)
cupsd    12207 root    7u  IPv4 93264      0t0  TCP localhost:ipp (LISTEN)
root@msi-pro:~# lsof -i :993
root@msi-pro:~# lsof -i :995

```

Then, the following command was executed to list open files along with the processes who opened them under amin's home directory:

```

amin@msi-pro:~$ sudo lsof -u amin > dummy.txt
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
Output information may be incomplete.
amin@msi-pro:~$ less dummy.txt

```

The following are processes who had opened files under amin's home directory:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
systemd	1547	amin	cwd	DIR	8,17	4096	2	/
systemd	1547	amin	rtd	DIR	8,17	4096	2	/
systemd	1547	amin	txt	REG	8,17	92544	26741576	/usr/lib/systemd/systemd
systemd	1547	amin	mem	REG	8,17	629384	26739313	/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0.11.2
systemd	1547	amin	mem	REG	8,17	911904	26741003	/usr/lib/x86_64-linux-gnu/libm.so.6
systemd	1547	amin	mem	REG	8,17	4730136	26739102	/usr/lib/x86_64-linux-gnu/libcrypto.so.3
systemd	1547	amin	mem	REG	8,17	157768	26739075	/usr/lib/x86_64-linux-gnu/libgpg-error.so.0.33.1
systemd	1547	amin	mem	REG	8,17	30704	26739055	/usr/lib/x86_64-linux-gnu/libcap-ng.so.0.0.0
systemd	1547	amin	mem	REG	8,17	190456	26739109	/usr/lib/x86_64-linux-gnu/liblzma.so.5.4.1
systemd	1547	amin	mem	REG	8,17	763816	26740887	/usr/lib/x86_64-linux-gnu/libzstd.so.1.5.4
systemd	1547	amin	mem	REG	8,17	149952	26739091	/usr/lib/x86_64-linux-gnu/libltdl.so.7.9.4

As shown below, "inotify" appears in the output, indicating filesystem watching:

systemd	1547	amin	9u	a_inode	0,14	0	55	[eventpoll:10,13]
systemd	1547	amin	10r	REG	0,21	0	23756	/proc/1547/mountinfo
systemd	1547	amin	11r	a_inode	0,14	0	55	inotify
systemd	1547	amin	12u	unix	0x00000000bfffcd57	0t0	19657	/run/user/1000/bus type=STREAM (LISTEN)
systemd	1547	amin	13r	a_inode	0,14	0	55	inotify
systemd	1547	amin	14r	REG	0,21	0	4026532077	/proc/swaps
systemd	1547	amin	15u	netlink		0t0	23757	KOBJECT_UEVENT
systemd	1547	amin	16u	unix	0x0000000013bf2b54	0t0	23761	/run/user/1000/systemd/notify type=DGRAM (UNCONN)

Then, the following command is run to list all open files in /tmp directory along with the processes who had opened them:

```
amin@msi-pro:~$ sudo lsof +D /tmp
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
Output information may be incomplete.
COMMAND  PID USER  FD  TYPE    DEVICE  SIZE/OFF  NODE NAME
Xorg      991 root   8u   unix 0x00000000110e66ac  0t0    15719 /tmp/.X11-unix/X0 type=STREAM (LISTEN)
x-session 1576 amin  14u   unix 0x00000000b8a16cd2  0t0    25269 /tmp/.ICE-unix/1576 type=STREAM (LISTEN)
ssh-agent 1640 amin   3u   unix 0x000000000316141f  0t0    33829 /tmp/ssh-c0B0oynd8TGx/agent.1576 type=STREAM (LISTEN)
brave     2621 amin  15u   unix 0x0000000008a430965  0t0    23976 /tmp/.org.chromium.Chromium.cplqRC/SingletonSocket type=STREAM (LISTEN)
bash     20396 amin   cwd   DIR      8,17    4096  22544385 /tmp
sudo     27502 root   cwd   DIR      8,17    4096  22544385 /tmp
sudo     27503 root   cwd   DIR      8,17    4096  22544385 /tmp
systemctl 27504 root   cwd   DIR      8,17    4096  22544385 /tmp
systemctl 27504 root   txt   REG      8,17   6567104  22544389 /tmp/victim
systemctl 27504 root   3r    REG      8,17    192  22544398 /tmp/victim.json
amin@msi-pro:~$
```

As shown above, two suspicious files /tmp/victim and /tmp/victim.json files are open.

Then, the following command was executed to list all open files in `/var` directory along with the processes who opened them:

```
amin@msi-pro:~$ sudo lsof +D /var
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
Output information may be incomplete.
COMMAND      PID       USER    FD   TYPE DEVICE SIZE/OFF      NODE NAME
systemd-j    415       root    mem   REG  8,17  8388608 24915445 /var/log/journal/c2d73a2987b64d7db0bab0eafbd34d2e/user-1000.j
ournal
systemd-j    415       root    mem   REG  8,17  8388608 24915414 /var/log/journal/c2d73a2987b64d7db0bab0eafbd34d2e/system.jour
nal
systemd-j    415       root    25u   REG  8,17  8388608 24915414 /var/log/journal/c2d73a2987b64d7db0bab0eafbd34d2e/system.jour
nal
systemd-j    415       root    48u   REG  8,17  8388608 24915445 /var/log/journal/c2d73a2987b64d7db0bab0eafbd34d2e/user-1000.j
ournal
auditd       845       root     5w   REG  8,17  1013239 24916520 /var/log/audit/audit.log
```

The next subsection explains how `netstat` command was used to analyze network statistics.

netstat

The following command was executed to list all open ports:

```
amin@msi-pro:~$ netstat --all
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:ssh              0.0.0.0:*               LISTEN
tcp        0      0 localhost:smtp           0.0.0.0:*               LISTEN
tcp        0      0 localhost:ipp            0.0.0.0:*               LISTEN
tcp        0      0 msi-pro:57554            10.0.0.3:ssh            ESTABLISHED
tcp        0      0 msi-pro:33270            sea09s30-in-f14.1:https TIME_WAIT
```

Then, the following command was executed to list all listening ports:

```
amin@msi-pro:~$ netstat --listening
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:ssh              0.0.0.0:*               LISTEN
tcp        0      0 localhost:smtp           0.0.0.0:*               LISTEN
tcp        0      0 localhost:ipp            0.0.0.0:*               LISTEN
tcp6       0      0 localhost:smtp           [::]:*                  LISTEN
tcp6       0      0 localhost:ipp            [::]:*                  LISTEN
```

Then, the following command was executed to list all TCP occupied ports:

```
amin@msi-pro:~$ netstat --tcp
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 msi-pro:57554            10.0.0.3:ssh            ESTABLISHED
tcp        0      8 msi-pro:27844            10.0.0.3:48247          ESTABLISHED
tcp6       0      0 msi-pro:34838            2600:9000:26ce:5a:https ESTABLISHED
amin@msi-pro:~$
```

As shown above, there is a TCP connection on port 48247 from host 10.0.0.3 to port 27844 on our machine. Note that 48247 is a registered port.

Then the following command was executed to list PID and name of programs who have opened internet ports at that moment:

```
amin@msi-pro:~$ netstat --program
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 msi-pro:57554          10.0.0.3:ssh            ESTABLISHED 20482/ssh
tcp        0      8 msi-pro:27844          10.0.0.3:48247          ESTABLISHED -
tcp6       40      0 msi-pro:34838          2600:9000:26ce:5a:https CLOSE_WAIT 2675/brave --type=u
udp        0      0 msi-pro:bootpc         _gateway:bootps        ESTABLISHED -
udp6       0      0 msi-pro:35802          sea15s07-in-x0e.1:https ESTABLISHED 2675/brave --type=u
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags   Type       State      I-Node  PID/Program name  Path
```

Then the following command was executed to list all processes with active network connection for a continuous period of time:

```
amin@msi-pro:~$ netstat --continuous
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 msi-pro:57554          10.0.0.3:ssh            ESTABLISHED
tcp        0      8 msi-pro:27844          10.0.0.3:48247          ESTABLISHED
tcp6       40      0 msi-pro:34838          2600:9000:26ce:5a:https CLOSE_WAIT
tcp6       0      0 msi-pro:48778          sea09s30-in-x03.1:https ESTABLISHED
udp        0      0 msi-pro:bootpc         _gateway:bootps        ESTABLISHED
udp6       0      0 msi-pro:48373          sea09s30-in-x0e.1:https ESTABLISHED
udp6       0      0 msi-pro:35802          sea15s07-in-x0e.1:https ESTABLISHED
Active UNIX domain sockets (w/o servers)
```

Then, the following command was executed to list all network routes without converting the numeric addresses to symbolic names:

```
amin@msi-pro:~$ netstat --route --numeric
Kernel IP routing table
Destination      Gateway          Genmask          Flags   MSS Window  irtt Iface
0.0.0.0          10.0.0.1         0.0.0.0          UG      0 0        0 wlp34s0
10.0.0.0         0.0.0.0          255.255.255.0    U       0 0        0 wlp34s0
amin@msi-pro:~$
```

Then, the following command was executed:

```
amin@msi-pro:~$ sudo netstat -tulpne
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       User        Inode      PID
/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      0           31093      990
/sshd: /usr/sbin
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      0           21145      153
6/exim4
```

The next subsection explains how the *top* command was used to list processes on msi-pro workstation.

top

The following command was executed to display dynamic real-time information about running processes:

```
amin@msi-pro:~$ top
```

The following is the output of the previous command:

```
amin@msi-pro:~$ top
top - 08:10:10 up 9:38, 2 users, load average: 1.54, 0.87, 0.46
Tasks: 342 total, 1 running, 341 sleeping, 0 stopped, 0 zombie
%Cpu(s): 3.4 us, 0.5 sy, 0.0 ni, 96.1 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 31979.3 total, 1083.8 free, 4938.0 used, 26699.2 buff/cache
MiB Swap: 976.0 total, 958.7 free, 17.2 used. 27041.3 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 33181 amin      20   0 1392.3g 71188 51820 S   0.0   0.2   0:00.02 brave
 33246 root       20   0     0     0     0 I   0.0   0.0   0:00.00 kworker/11:0
 33253 root       20   0     0     0     0 I   0.0   0.0   0:00.00 kworker/13:3-events
 33349 root       20   0     0     0     0 I   0.0   0.0   0:00.00 kworker/3:0-mm_percpu_wq
 33350 root       20   0     0     0     0 I   0.0   0.0   0:00.00 kworker/10:0-rcu_par_gp
 33356 root       20   0     0     0     0 I   0.0   0.0   0:00.00 kworker/2:1
 33412 root       20   0  10072  4808  4324 S   0.0   0.0   0:00.02 sudo
 33413 root       20   0  10072   528     0 S   0.0   0.0   0:00.00 sudo
 33414 root       20   0 1763076 19324 6228 S   0.0   0.1   0:00.04 systemctl3.4
 33434 root       20   0     0     0     0 I   0.0   0.0   0:00.00 kworker/u64:1-events_unbound
```

As shown there is a process called “systemctl3.4” that is not associated with any system services. The next subsection explains how *chkrootkit* was used to check for rootkits.

Chkrootkit

The following command was executed to check system binaries for malicious modifications as follows:

```
amin@msi-pro:~$ sudo chkrootkit
[sudo] password for amin:
ROOTDIR is '/'
Checking `amd'... not found
Checking `basename'... not infected
Checking `biff'... not found
Checking `chfn'... not infected
Checking `chsh'... not infected
Checking `cron'... not infected
Checking `crontab'... not infected
Checking `date'... not infected
Checking `du'... not infected
Checking `dirname'... not infected
Checking `echo'... not infected
Checking `egrep'... not infected
Checking `env'... not infected
Checking `find'... not infected
Checking `fingerd'... not found
Checking `gpm'... not found
Checking `grep'... not infected
```

Chkrootkit seems to complain about libreoffice and ruby as shown below:

```
WARNING: The following suspicious files and directories were found:
/usr/lib/libreoffice/share/.registry
/usr/lib/ruby/vendor_ruby/rubygems/ssl_certs/.document
/usr/lib/ruby/vendor_ruby/rubygems/tsort/.document
/usr/lib/ruby/vendor_ruby/rubygems/optparse/.document
/usr/lib/ruby/gems/3.1.0/gems/typeprof-0.21.2/vscode/.vscodeignore
/usr/lib/ruby/gems/3.1.0/gems/typeprof-0.21.2/vscode/.vscode
/usr/lib/ruby/gems/3.1.0/gems/typeprof-0.21.2/vscode/.gitignore
```

Also, *Chkrootkit* found one suspicious file called `/tmp/victim` as shown below:

```
WARNING: Possible Linux.Xor.DDoS installed:
/tmp/victim
```

Also, *Chkrootkit* complains since the network interface is in promiscuous mode as shown below:

```
WARNING: Output from ifpromisc:
lo: not promisc and no packet sniffer sockets
wlp34s0: PACKET SNIFFER(/tmp/victim[33088], /tmp/victim[33088], /usr/sbin/NetworkManager[894], /usr/sbin/NetworkManager[894], /usr/sbin/wpa_supplicant[895])
```

At this point, we were certain that the process named “systemctl3.4”, and the binary file `/tmp/victim` were suspicious and possibly malicious. The next subsection explains how *Rkhunter* was used to check for rootkits.

Rkhunter

The following command was executed to check the version of *Rkhunter*:

```
amin@msi-pro:~$ sudo rkhunter -V
Rootkit Hunter 1.4.6

This software was developed by the Rootkit Hunter project team.
Please review your rkhunter configuration files before using.
Please review the documentation before posting bug reports or questions.
To report bugs, provide patches or comments, please go to:
http://rkhunter.sourceforge.net

To ask questions about rkhunter, please use the rkhunter-users mailing list.
Note this is a moderated list: please subscribe before posting.

Rootkit Hunter comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it under the
terms of the GNU General Public License. See the LICENSE file for details.

amin@msi-pro:~$
```

Rkhunter's config file needed some tweaks, so we opened it as shown below:

```
amin@msi-pro:~$ sudo vim /etc/rkhunter.conf
```

WEB_CMD was Located in *Rkhunter*'s config file as shown below:

```
File Edit View Search Terminal Help
# everything written to stderr is ignored. For example:
#
#   WEB_CMD="/opt/bin/dlfile --timeout 5m -q"
#
# *BSD users may want to use the 'ftp' command, provided that it supports the
# HTTP protocol:
#
#   WEB_CMD="ftp -o -"
#
# This option has no default value.
#
WEB_CMD="/bin/false"
#
# Set the following option to '1' if locking is to be used when rkhunter runs.
# The lock is set just before logging starts, and is removed when the program
# ends. It is used to prevent items such as the log file, and the file
# properties file, from becoming corrupted if rkhunter is running more than
1190,20 88%
```

Value of WEB_CMD was change on *Rkhunter*'s config file as shown below:

```
File Edit View Search Terminal Help
# everything written to stderr is ignored. For example:
#
#     WEB_CMD="/opt/bin/dlfile --timeout 5m -q"
#
# *BSD users may want to use the 'ftp' command, provided that it supports the
# HTTP protocol:
#
#     WEB_CMD="ftp -o -"
#
# This option has no default value.
#
WEB_CMD=/usr/bin/wget
#
# Set the following option to '1' if locking is to be used when rkhunter runs.
# The lock is set just before logging starts, and is removed when the program
# ends. It is used to prevent items such as the log file, and the file
# properties file, from becoming corrupted if rkhunter is running more than
"/etc/rkhunter.conf" 1352L, 49229B written 1190,21 88%
```

Rkhunter's database was updated as shown below:

```
amin@msi-pro:~$ sudo rkhunter --update
[ Rootkit Hunter version 1.4.6 ]

Checking rkhunter data files...
  Checking file mirrors.dat [ Skipped ]
  Checking file programs_bad.dat [ Update failed ]
  Checking file backdoorports.dat [ Update failed ]
  Checking file suspscan.dat [ Update failed ]
  Checking file i18n versions [ Update failed ]

Please check the log file (/var/log/rkhunter.log)

amin@msi-pro:~$
```

There was an issue updating *Rkhunter*'s database, so *Rkhunter*'s log file was opened as shown below:

```
amin@msi-pro:~$ sudo less /var/log/rkhunter.log
```

The following is the content of *Rkhunter*'s log file:

```
[16:42:17] Info: The mirrors file will not be updated
[16:42:17] Info: Logging to log file: /var/log/rkhunter.log
[16:42:17] Info: Locking is not being used
[16:42:17]
[16:42:17] Checking rkhunter data files...
[16:42:17] Info: Created temporary file '/var/lib/rkhunter/tmp/rkhunter.upd.82CX9YVRUf'
[16:42:17] Checking file mirrors.dat [ Skipped ]
[16:42:17] Info: The mirrors file has no required mirrors in it: /var/lib/rkhunter/db/mirrors.dat
[16:42:17] Warning: Download of 'programs_bad.dat' failed: Unable to determine the latest version number.
[16:42:17] Checking file programs_bad.dat [ Update failed ]
[16:42:17] Info: The mirrors file has no required mirrors in it: /var/lib/rkhunter/db/mirrors.dat
[16:42:17] Warning: Download of 'backdoorports.dat' failed: Unable to determine the latest version number
.
[16:42:17] Checking file backdoorports.dat [ Update failed ]
[16:42:17] Info: The mirrors file has no required mirrors in it: /var/lib/rkhunter/db/mirrors.dat
[16:42:17] Warning: Download of 'suspscan.dat' failed: Unable to determine the latest version number.
[16:42:17] Checking file suspscan.dat [ Update failed ]
[16:42:17] Info: The mirrors file has no required mirrors in it: /var/lib/rkhunter/db/mirrors.dat
[16:42:17] Checking file i18n versions [ Update failed ]
[16:42:17] Warning: Download of 'i18n.ver' failed: Unable to determine the latest version number.
[16:42:17]
[16:42:17] Info: End date is Sat May 31 04:42:17 PM PDT 2025
(END)
```

As shown above, *Rkhunter* complained that it did not have any mirrors listed to download its database as shown below:

So *Rkhunter*'s mirrors list file was opened as shown below

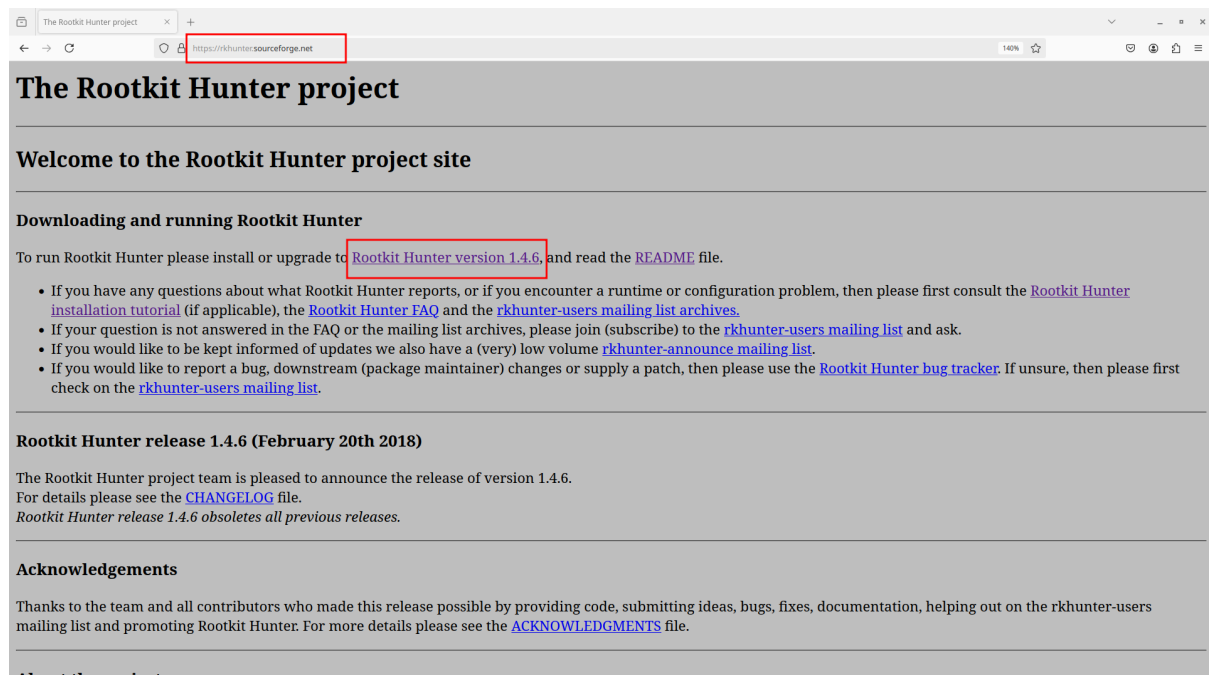
```
amin@msi-pro:~$ sudo vim /var/lib/rkhunter/db/mirrors.dat
```

The following was the content of *Rkhunter*'s mirrors list file:

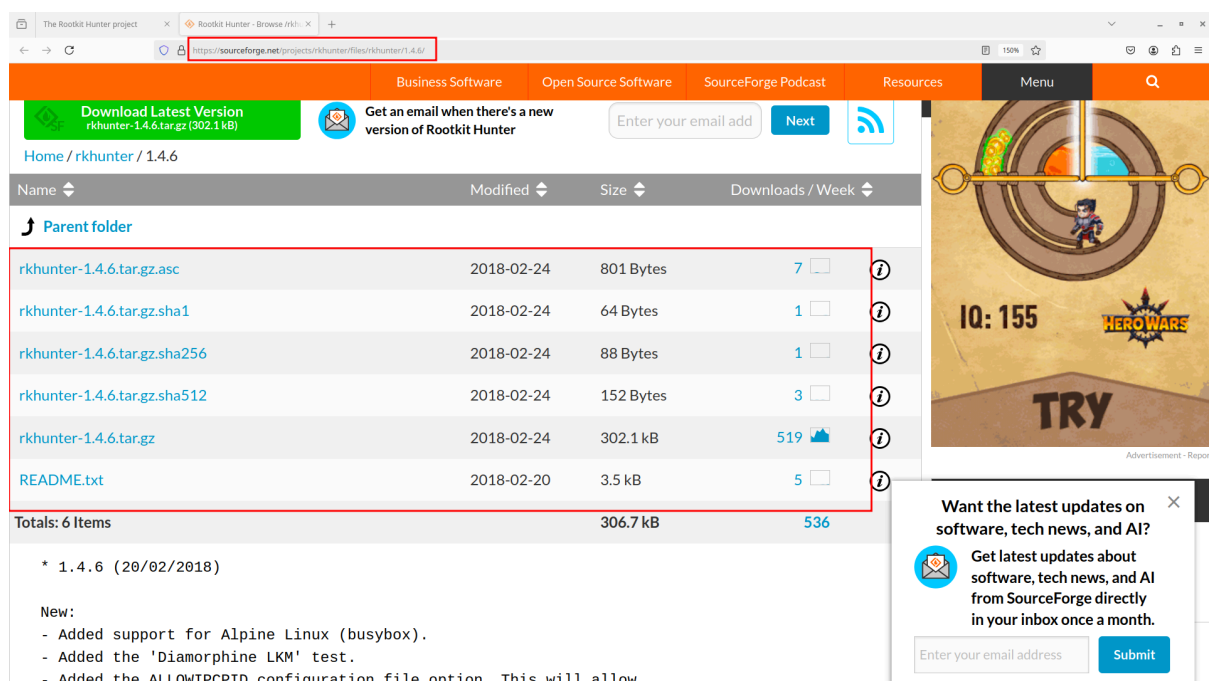
```
Version:2007060601
mirror=http://rkhunter.sourceforge.net
mirror=http://rkhunter.sourceforge.net
~
```

As shown above, the mirrors were clearly listed, so the problem was something else.

We worked around the problem by trying to download the latest database from *Rkhunter's* database. Rkhunter's homepage was visited to download the latest database. The link shown below was clicked to do so:



After clicking of the corresponding link, the following web page should pop up:



rkhunter-<version>.tar.gz was download as shown below:

Download Latest Version
rkhunter-1.4.6.tar.gz (302.1 kB)

Get an email when there's a new version of Rootkit Hunter

Home / rkhunter / 1.4.6

Name	Modified	Size	Downloads / Week
Parent folder			
rkhunter-1.4.6.tar.gz.asc	2018-02-24	801 Bytes	7
rkhunter-1.4.6.tar.gz.sha1	2018-02-24	64 Bytes	1
rkhunter-1.4.6.tar.gz.sha256	2018-02-24	88 Bytes	1
rkhunter-1.4.6.tar.gz.sha512	2018-02-24	152 Bytes	3
rkhunter-1.4.6.tar.gz	2018-02-24	302.1 kB	519
README.txt	2018-02-20	3.5 kB	5
Totals: 6 Items		306.7 kB	536

* 1.4.6 (20/02/2018)

New:

- Added support for Alpine Linux (busybox).
- Added the 'Diamorphine LKM' test.
- Added the ALLOWPCPID configuration file option. This will allow specific PIDs to be whitelisted from the shared memory check.
- Added the ALLOWPCUSER configuration file option. This will allow specific usernames to be whitelisted from the shared memory check.
- Added the IPC_SEG_SIZE configuration file option. This can be used to set the minimum shared memory segment size to check. The default value is 1048576 bytes (1MB).
- Added the SHM_CHECK configuration file option. Setting this

Recommended Projects

- Open Source Tripwire®
"Open Source Tripwire® has now moved to https://..."
- Unhide
Unhide is a forensic tool to find processes and TCP/UDP port...
- zeppoo
Zeppoo allows you to detect rootkits on i386 and x86_64...
- Brakeman
A static analysis security vulnerability scanner for Rub...

YOU HAVE ONLY 2 MOVES
IQ: 155

The content of rkhunter-<version>.tar.gz was extracted as shown below:

```
amin@msi-pro:~/Downloads$ tar -xzf ./rkhunter-1.4.6.tar.gz
rkhunter-1.4.6/
rkhunter-1.4.6/installer.sh
rkhunter-1.4.6/files/
rkhunter-1.4.6/files/stat.pl
rkhunter-1.4.6/files/LICENSE
rkhunter-1.4.6/files/suspscan.dat
rkhunter-1.4.6/files/rkhunter.conf
```

We navigated to the directory containing the content of rkhunter-<version>.[tar.gz](#) as follows:

```
amin@msi-pro: ~/Downloads/rkhunter-1.4.6/files
amin@msi-pro:~/Downloads$ cd rkhunter-1.4.6/
amin@msi-pro:~/Downloads/rkhunter-1.4.6$ ls
files  installer.sh
amin@msi-pro:~/Downloads/rkhunter-1.4.6$ cd files/
amin@msi-pro:~/Downloads/rkhunter-1.4.6/files$ ls
ACKNOWLEDGMENTS  FAQ  programs_bad.dat  rkhunter.conf
backdoorports.dat  filehashsha.pl  readlink.sh  rkhunter.spec
CHANGELOG  i18n  README  signatures
check_modules.pl  LICENSE  rkhunter  stat.pl
contrib  mirrors.dat  rkhunter.8  suspscan.dat
amin@msi-pro:~/Downloads/rkhunter-1.4.6/files$
```

Then the following command was executed to update *Rkhunter's* database as shown below:

```
amin@msi-pro:~/Downloads/rkhunter-1.4.6/files$ sudo ./rkhunter --propupd
[ Rootkit Hunter version 1.4.6 ]
File updated: searched for 181 files, found 143
amin@msi-pro:~/Downloads/rkhunter-1.4.6/files$
```

Then the following command was executed to check for rootkits, backdoors, and possible local exploits:

```
amin@msi-pro:~/Downloads/rkhunter-1.4.6/files$ sudo ./rkhunter --check
[ Rootkit Hunter version 1.4.6 ]

Checking system commands...

Performing 'strings' command checks
Checking 'strings' command [ OK ]

Performing 'shared libraries' checks
Checking for preloading variables [ None found ]
Checking for preloaded libraries [ None found ]
Checking LD_LIBRARY_PATH variable [ Not found ]

Performing file properties checks
Checking for prerequisites [ OK ]
/usr/sbin/adduser [ OK ]
/usr/sbin/chroot [ OK ]
/usr/sbin/cron [ OK ]
/usr/sbin/depmod [ OK ]
/usr/sbin/fsck [ OK ]
/usr/sbin/groupadd [ OK ]
/usr/sbin/groupdel [ OK ]
/usr/sbin/groupmod [ OK ]
/usr/sbin/grpck [ OK ]
```

File `/usr/bin/lwp-request` seemed to have a problem as shown below:

```
/usr/bin/nmmt [ OK ]
/usr/bin/kmod [ OK ]
/usr/bin/systemd [ OK ]
/usr/bin/systemctl [ OK ]
/usr/bin/mawk [ OK ]
/usr/bin/lwp-request [ Warning ]
/usr/bin/bsd-mailx [ OK ]
/usr/bin/dash [ OK ]
/usr/bin/x86_64-linux-gnu-size [ OK ]
```

Also, *Rkhunter* checked for rootkits as shown below:

```
Checking for rootkits...

Performing check of known rootkit files and directories
55808 Trojan - Variant A [ Not found ]
ADM Worm [ Not found ]
AjaKit Rootkit [ Not found ]
Adore Rootkit [ Not found ]
aPa Kit [ Not found ]
Apache Worm [ Not found ]
Ambient (ark) Rootkit [ Not found ]
Balaaur Rootkit [ Not found ]
BeastKit Rootkit [ Not found ]
beX2 Rootkit [ Not found ]
BOBKit Rootkit [ Not found ]
cb Rootkit [ Not found ]
CiNIK Worm (Slapper.B variant) [ Not found ]
Danny-Boy's Abuse Kit [ Not found ]
Devil RootKit [ Not found ]
Diamorphine LKM [ Not found ]
Dica-Kit Rootkit [ Not found ]
Dreams Rootkit [ Not found ]
Duarawkz Rootkit [ Not found ]
Ebury backdoor [ Not found ]
Enye LKM [ Not found ]
```

In addition, *Rkhunter* warned us about suspiciously large memory segments as shown below:

```
Performing malware checks
Checking running processes for suspicious files [ None found ]
Checking for login backdoors [ None found ]
Checking for sniffer log files [ None found ]
Checking for suspicious directories [ None found ]
Checking for suspicious (large) shared memory segments [ Warning ]
Checking for Apache backdoor [ Not found ]
```

Then, *Rkhunter* checked for network anomalies as shown below:

```
Checking the network...

Performing checks on the network ports
Checking for backdoor ports [ None found ]

Performing checks on the network interfaces
Checking for promiscuous interfaces [ None found ]

Checking the local host
```

Then, *Rkhunter* checked for local system anomalies as shown below:

```
Checking the local host...

Performing system boot checks
  Checking for local host name           [ Found ]
  Checking for system startup files       [ Found ]
  Checking system startup files for malware [ None found ]

Performing group and account checks
  Checking for passwd file                [ Found ]
  Checking for root equivalent (UID 0) accounts [ None found ]
  Checking for passwordless accounts      [ None found ]
  Checking for passwd file changes        [ Warning ]
  Checking for group file changes         [ Warning ]
  Checking root account shell history files [ OK ]

Performing system configuration file checks
  Checking for an SSH configuration file   [ Found ]
  Checking if SSH root access is allowed   [ Warning ]
  Checking if SSH protocol v1 is allowed   [ Not set ]
  Checking for other suspicious configuration settings [ None found ]
  Checking for a running system logging daemon [ Found ]
  Checking for a system logging configuration file [ Found ]

Performing filesystem checks
```

As shown above, Rkhunter complained about three things:

- 1) Root SSH access being allowed
- 2) Suspicious /etc/passwd changes
- 3) Suspicious /etc/group changes

When *Rkhunter* was done, it display a summary segment shown below:

```
System checks summary
=====

File properties checks...
  Files checked: 143
  Suspect files: 1

Rootkit checks...
  Rootkits checked : 498
  Possible rootkits: 17

Applications checks...
  All checks skipped

The system checks took: 1 minute and 8 seconds

All results have been written to the log file: /var/log/rkhunter.log

One or more warnings have been found while checking the system.
Please check the log file (/var/log/rkhunter.log)
```

Rkhunter pointed us to its log file, and we opened its log file as shown below:

```
amin@msi-pro: ~/Downloads/rkhunter-1.4.6/files$ sudo vim /var/log/rkhunter.log
```

The following is the content of Rkhunter's log file:

```
File Edit View Search Terminal Help
[17:10:53] Running Rootkit Hunter version 1.4.6 on msi-pro
[17:10:53]
[17:10:53] Info: Start date is Sat May 31 05:10:53 PM PDT 2025
[17:10:53]
[17:10:53] Checking configuration file and command-line options...
[17:10:53] Info: Detected operating system is 'Linux'
[17:10:53] Info: Found O/S name: Debian GNU/Linux 12 (bookworm)
[17:10:53] Info: Command line is ./rkhunter --check
[17:10:53] Info: Environment shell is /bin/bash; rkhunter is using dash
[17:10:53] Info: Using configuration file '/etc/rkhunter.conf'
[17:10:53] Info: Installation directory is '/usr'
[17:10:53] Info: Using language 'en'
[17:10:53] Info: Using '/var/lib/rkhunter/db' as the database directory
[17:10:53] Info: Using '/usr/share/rkhunter/scripts' as the support script directory
[17:10:53] Info: Using '/usr/local/sbin /usr/local/bin /usr/sbin /usr/bin /sbin /bin /usr/libexec' as the
command directories
[17:10:53] Info: Using '/var/lib/rkhunter/tmp' as the temporary directory
[17:10:53] Info: No mail-on-warning address configured
[17:10:53] Info: X will be automatically detected
[17:10:53] Info: Using second color set
[17:10:53] Info: Found the 'basename' command: /usr/bin/basename
[17:10:53] Info: Found the 'diff' command: /usr/bin/diff
[17:10:53] Info: Found the 'dirname' command: /usr/bin/dirname
1,1 Top
```

The next subsection explains how AIDE was used to check for filesystem integrity.

AIDE

In order to check for file system anomalies, Advanced Intrusion Detection Environment (AIDE) was used.

The following command was used to initialize *AIDE*:

```
amin@msi-pro:/etc/aide$ sudo aideinit
Overwrite existing /var/lib/aide/aide.db.new [Yn]? y
Running aide --init...
```

Then, *AIDE* created a new snapshot of the filesystem as shown below:

```
amin@msi-pro:~$ sudo ls -lh /var/lib/aide/aide.db.new
-rw----- 1 _aide _aide 0 May 31 17:23 /var/lib/aide/aide.db.new
amin@msi-pro:~$
```

This newly created snapshot of the filesystem was used as our baseline, so we moved it to the location where *AIDE* expects the baseline filesystem snapshot as shown below:

```
amin@msi-pro:/etc/aide$ sudo mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db
amin@msi-pro:/etc/aide$
```

```
amin@msi-pro: /etc/aide$ sudo vim /etc/aide/aide.conf
```

```
SerMemberDELog = Full+ANF
LinkedLog = Log-n

# include rules from /etc/aide/aide.conf.d, honoring the
# file name restrictions that are common in Debian. Files with
# the x bit set are executed and their output included.
@@x_include /etc/aide/aide.conf.d ^[a-zA-Z0-9_-]+$

/etc ActLog
/bin ActLog
/sbin ActLog
/usr ActLog
/home ActLog

~
~
~
~
~
~
~
```

```

amin@msi-pro:/etc/aide$ sudo aide --config=/etc/aide/aide.conf --check
WARNING: hash calculation: '/etc/aide/.aide.conf.swp' has been changed (changed attributes: c+m, hash could not be calculated)
WARNING: hash calculation: '/home/amin/.cache/BraveSoftware/Brave-Browser/Default/Cache/Cache_Data/index-dir/the-real-index' h
as been changed (changed attributes: s+c+m+i, hash could not be calculated)
WARNING: hash calculation: '/home/amin/.config/BraveSoftware/Brave-Browser/Local State' has been changed (changed attributes:
c+m+i, hash could not be calculated)
WARNING: get_file_status: lstat() failed for /run/user/1000/doc: Permission denied
WARNING: get_file_status: lstat() failed for /run/user/1000/gvfs: Permission denied
WARNING: hash calculation: '/home/amin/.config/BraveSoftware/Brave-Browser/Default/Cookies-journal' has been changed (changed
attributes: c+m, hash could not be calculated)
WARNING: hash calculation: '/home/amin/.config/BraveSoftware/Brave-Browser/Default/Preferences' has been changed (changed attr
ibutes: c+m+i, hash could not be calculated)
WARNING: hash calculation: '/home/amin/.config/BraveSoftware/Brave-Browser/Default/Cookies' has been changed (changed attribut
es: c+m, hash could not be calculated)
WARNING: hash calculation: '/home/amin/.config/BraveSoftware/Brave-Browser/Safe Browsing/UrlMalBin.store' has been changed (ch
anged attributes: c+m+i, hash could not be calculated)
WARNING: tried to read access ACL on /home/amin/.config/BraveSoftware/Brave-Browser/Safe Browsing/UrlSoceng.store.4_1339321364
6631937 but failed with: No such file or directory
WARNING: tried to read default ACL on /home/amin/.config/BraveSoftware/Brave-Browser/Safe Browsing/UrlSoceng.store.4_133932136
46631937 but failed with: Invalid argument
WARNING: listxattrs failed for /home/amin/.config/BraveSoftware/Brave-Browser/Safe Browsing/UrlSoceng.store.4_1339321364663193
7: No such file or directory
WARNING: lgetfilecon_raw failed for /home/amin/.config/BraveSoftware/Brave-Browser/Safe Browsing/UrlSoceng.store.4_13393213646
631937: No such file or directory
WARNING: hash calculation: open() failed for /home/amin/.config/BraveSoftware/Brave-Browser/Safe Browsing/UrlSoceng.store.4_13
393213646631937: No such file or directory (hashsums could not be calculated)

```

32

Auditd

In addition to *AIDE*, in order to check for file system anomalies, *Auditd* was used.

We ran the following commands to install and initialize *Auditd*:

```
sudo apt install auditd audispd-plugins
sudo systemctl status auditd
```

Then, the following command was executed to check /tmp directory using *Auditd*:

```
amin@msi-pro:/etc/aide$ sudo auditctl -w /tmp
amin@msi-pro:/etc/aide$
```

Then, the following command was executed to show summary result of *Auditd*:

```
amin@msi-pro:/etc/aide$ sudo aureport --summary

Summary Report
=====
Range of time in logs: 05/31/2025 01:24:17.060 - 05/31/2025 18:54:53.912
Selected time for report: 05/31/2025 01:24:17 - 05/31/2025 18:54:53.912
Number of changes in configuration: 16
Number of changes to accounts, groups, or roles: 8
Number of logins: 2
Number of failed logins: 0
Number of authentications: 25
Number of failed authentications: 7
Number of users: 4
Number of terminals: 23
Number of host names: 2
Number of executables: 15
Number of commands: 11
Number of files: 7
Number of AVC's: 0
Number of MAC events: 0
Number of failed syscalls: 0
Number of anomaly events: 8
Number of responses to anomaly events: 0
Number of crypto events: 0
Number of integrity events: 0
Number of virt events: 0
Number of keys: 0
```

Then, the following command was executed to check for unusual logins to our system:

```
amin@msi-pro:/etc/aide$ sudo aureport -l

Login Report
=====
# date time auid host term exe success event
=====
1. 05/31/2025 03:09:30 1000 msi-pro /dev/tty7 /usr/sbin/lightdm yes 98
2. 05/31/2025 03:14:30 1000 msi-pro /dev/tty7 /usr/sbin/lightdm yes 104
amin@msi-pro:/etc/aide$
```

Then, the following command was used to check for unusual execution of binaries:

```
amin@msi-pro:/etc/aide$ sudo aureport -x

Executable Report
=====
# date time exe term host auid event
=====
1. 05/31/2025 01:24:17 /usr/sbin/auditctl (none) ? -1 30
2. 05/31/2025 01:24:17 /usr/sbin/auditctl (none) ? -1 31
3. 05/31/2025 01:24:17 /usr/sbin/auditctl (none) ? -1 32
4. 05/31/2025 01:24:17 /usr/lib/systemd/systemd ? ? -1 33
5. 05/31/2025 01:24:17 /usr/bin/sudo /dev/pts/1 ? 1000 34
6. 05/31/2025 01:24:17 /usr/bin/sudo /dev/pts/1 ? 1000 35
7. 05/31/2025 01:24:50 /usr/bin/sudo /dev/pts/1 ? 1000 36
8. 05/31/2025 01:24:50 /usr/bin/sudo pts/1 ? 1000 37
9. 05/31/2025 01:24:50 /usr/bin/sudo /dev/pts/1 ? 1000 38
10. 05/31/2025 01:24:50 /usr/bin/sudo /dev/pts/1 ? 1000 39
```

Then, the following command was used to check for unusual file changes:

```
amin@msi-pro:/etc/aide$ sudo aureport -f

File Report
=====
# date time file syscall success exe auid event
=====
1. 05/31/2025 18:50:34 /tmp/ 263 yes /opt/brave.com/brave/brave 1000 964
2. 05/31/2025 18:50:34 /tmp/ 257 yes /opt/brave.com/brave/brave 1000 965
3. 05/31/2025 18:50:34 /tmp/ 85 yes /opt/brave.com/brave/brave 1000 966
4. 05/31/2025 18:50:34 /tmp/.org.chromium.Chromium.zWYAX4 257 yes /usr/bin/cinnamon 1000 967
5. 05/31/2025 18:50:34 /tmp/ 257 yes /opt/brave.com/brave/brave 1000 968
6. 05/31/2025 18:50:34 /tmp/ 85 yes /opt/brave.com/brave/brave 1000 969
7. 05/31/2025 18:50:34 /tmp/ 263 yes /opt/brave.com/brave/brave 1000 970
8. 05/31/2025 18:50:34 /tmp/.org.chromium.Chromium.2sfGGy 257 yes /usr/bin/cinnamon 1000 971
9. 05/31/2025 18:51:03 /tmp/ 257 yes /tmp/victim 1000 984
10. 05/31/2025 18:51:24 /tmp/keylog.txt 257 yes /tmp/victim 1000 985
11. 05/31/2025 18:52:22 /tmp/ 257 yes /tmp/victim 1000 986
12. 05/31/2025 18:52:30 . 257 yes /usr/bin/ls 1000 987
13. 05/31/2025 18:55:01 /tmp/ 263 yes /opt/brave.com/brave/brave 1000 1020
```

Then, the following command was used to ensure user IDs were properly configured:

```
amin@msi-pro:/etc/aide$ sudo aureport -u

User ID Report
=====
# date time auid term host exe event
=====
1. 05/31/2025 01:24:17 -1 ? ? ? 5218
2. 05/31/2025 01:24:17 -1 (none) ? /usr/sbin/auditctl 30
3. 05/31/2025 01:24:17 -1 (none) ? /usr/sbin/auditctl 31
4. 05/31/2025 01:24:17 -1 (none) ? /usr/sbin/auditctl 32
5. 05/31/2025 01:24:17 -1 ? ? /usr/lib/systemd/systemd 33
6. 05/31/2025 01:24:17 1000 /dev/pts/1 ? /usr/bin/sudo 34
7. 05/31/2025 01:24:17 1000 /dev/pts/1 ? /usr/bin/sudo 35
8. 05/31/2025 01:24:50 1000 /dev/pts/1 ? /usr/bin/sudo 36
9. 05/31/2025 01:24:50 1000 pts/1 ? /usr/bin/sudo 37
10. 05/31/2025 01:24:50 1000 /dev/pts/1 ? /usr/bin/sudo 38
```

Then, the following command was executed to check the system for unusual system events:

```
amin@msi-pro:/etc/aide$ sudo aureport -e

Event Report
=====
# date time event type auid success
=====
1. 05/31/2025 01:24:17 5218 DAEMON_START -1 yes
2. 05/31/2025 01:24:17 30 CONFIG_CHANGE -1 yes
3. 05/31/2025 01:24:17 31 CONFIG_CHANGE -1 yes
4. 05/31/2025 01:24:17 32 CONFIG_CHANGE -1 yes
5. 05/31/2025 01:24:17 33 SERVICE_START -1 yes
6. 05/31/2025 01:24:17 34 USER_END 1000 yes
7. 05/31/2025 01:24:17 35 CRED_DISP 1000 yes
8. 05/31/2025 01:24:50 36 USER_ACCT 1000 yes
9. 05/31/2025 01:24:50 37 USER_CMD 1000 yes
10. 05/31/2025 01:24:50 38 CRED_REFR 1000 yes
11. 05/31/2025 01:24:50 39 USER_START 1000 yes
```

Just as *AIDE*, *Auditd*, did not give us any significant clues. Next subsection will explain malware' behavior.

Malware Behavior

Upon discovering that msi-pro machine had indeed been infected, and upon identifying /tmp/victim binary and the abnormal process's PID, we moved on to dissecting the malware and identifying its behaviour. Initially, strace command was used to list the system calls that the abnormal process was using. Majority of the system calls used by the abnormal process were network related such as socket(), bind(), and listen(), proving that the abnormal process was indeed communicating with a command and control server. Then, ltrace command was executed passing the suspicious binary; majority of the library calls were thread related, indicating that the suspicious binary had the capability to perform computationally intensive tasks simultaneously.

Moreover, using the /proc/[pid] directory, the suspicious process was dynamically analyzed. /proc/[pid]/comm file's value was "systemct3.4", the very same suspicious process name we noticed that had nothing to do with Debian after reviewing Debian's documentation. And also, /proc/[pid]/cmdline file's value was "victim", the name of the very same binary file that had raised suspicion. In addition, /proc/[pid]/cwd file's value was /tmp, giving an explanation as to why filesystem integrity tools such as AIDE and Auidtd had failed to detect suspicious file system modifications.

Also, going through the value of /proc/[pid]/fd, a file that stood out was, /tmp/victim.json. victim.json contained the MAC addresses and IPv4 addresses of msi-pro workstation and possibly host 10.0.0.3. Also, two port numbers were found in victim.json: 9999 and 8888. The very same port numbers that were generating the high number of half open TCP connections. The malware developers possibly were using this victim.json to configure the malware's victim. At this point, we were certain that the binary file "victim" and the abnormal process did in fact belong to the malware.. The following subsection explains how strace was used to identify the system calls the malicious process was using.

strace

The following command is used to list the system calls used by the suspicious process:

```
amin@msi-pro:~$ sudo strace --attach 33414
strace: Process 33414 attached
restart_syscall(<... resuming interrupted read ...>) = 1
clock_gettime(CLOCK_MONOTONIC, {tv_sec=34913, tv_nsec=268182785}) = 0
futex(0x85c980, FUTEX_WAKE_PRIVATE, 1) = 1
futex(0xc000064d28, FUTEX_WAKE_PRIVATE, 1) = 1
clock_gettime(CLOCK_MONOTONIC, {tv_sec=34913, tv_nsec=268457188}) = 0
futex(0xc000102948, FUTEX_WAKE_PRIVATE, 1) = 1
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=33414, si_uid=0} ---
rt_sigreturn({mask=[]}) = 1
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=33414, si_uid=0} ---
rt_sigreturn({mask=[]}) = 1
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=33414, si_uid=0} ---
rt_sigreturn({mask=[]}) = 1
```

The process mostly used networking system calls as listed below:

```
socket(AF_INET6, SOCK_STREAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 6
setsockopt(6, SOL_IPV6, IPV6_V6ONLY, [0], 4) = 0
```

```
bind(6, {sa_family=AF_INET6, sin6_port=htons(18182), sin6_flowinfo=htonl(0),
inet_pton(AF_INET6, "::", &sin6_addr), sin6_scope_id=0}, 28) = 0
listen(6, 4096)
getsockname(6, {sa_family=AF_INET6, sin6_port=htons(18182), sin6_flowinfo=htonl(0),
inet_pton(AF_INET6, "::", &sin6_addr), sin6_scope_id=0}, [112 => 28]) = 0
fcntl(17, F_SETPIPE_SZ, 1048576)
```

In order to get a summary of all system calls issued by the suspicious process, the following command was executed:

```
amin@msi-pro:~$ sudo strace --attach 33414 --summary-only
```

```
[sudo] password for amin:
```

```
strace: Process 33414 attached
```

```
^Cstrace: Process 33414 detached
```

% time	seconds	usecs/call	calls	errors	syscall
85.67	0.014595	540	27		futex
7.86	0.001339	29	46		clock_gettime
3.16	0.000538	179	3		nanosleep
1.61	0.000275	55	5		epoll_pwait
0.42	0.000072	18	4		write
0.27	0.000046	46	1	1	connect
0.21	0.000035	17	2		getsockname
0.19	0.000033	6	5		setsockopt
0.14	0.000024	8	3	1	read
0.12	0.000020	10	2		epoll_ctl
0.11	0.000018	18	1		socket
0.06	0.000011	11	1		close
0.06	0.000011	11	1		bind
0.05	0.000009	4	2		rt_sigreturn
0.03	0.000005	5	1		getpeername
0.03	0.000005	5	1		getsockopt
100.00	0.017036	162	105	2	total

As shown above, networking system calls were used heavily by the suspicious process. The next subsection explains how the *ltrace* command was used to list the dynamic library calls that the suspicious binary was using.

ltrace

The dynamic library calls of used by the suspicious binary are shown below:

```
amin@msi-pro: ~$ ltrace /tmp/victim
malloc(16) = 0x2545c2d0
pthread_attr_init(0x7fff362044a0, 0x2545c2d0, 0, 0x2545c2d0) = 0
pthread_self(0x7fff362044a0, 0x2545c2d0, 0, 0x2545c2d0) = 0x7fdc8df367c0
pthread_getattr_np(0x7fdc8df367c0, 0x7fff362044a0, 0, 0x2545c2d0) = 0
pthread_attr_getstack(0x7fff362044a0, 0x7fff36204490, 0x7fff36204498, 1) = 0
pthread_attr_destroy(0x7fff362044a0, 0x7fff36204490, 0x7ff000, 0x7fff36204498) = 0
free(0x2545c2d0) = <void>
mmap(0, 0x40000, 3, 34) = 0x7fdc8def6000
mmap(0, 0x20000, 0, 34) = 0x7fdc8ded6000
mmap(0, 0x100000, 0, 34) = 0x7fdc8ddd6000
mmap(0, 0x800000, 0, 34) = 0x7fdc8d5d6000
mmap(0, 0x4000000, 0, 34) = 0x7fdc895d6000
mmap(0, 0x20000000, 0, 34) = 0x7fdc695d6000
mmap(0, 0x20000000, 0, 34) = 0x7fdc495d6000
```

The following command was used to count the dynamic library calls of each individual library function as follows:

```
amin@msi-pro: ~$ ltrace -c /tmp/victim
run with sudo
% time      seconds  usecs/call   calls      function
-----
47.77      0.208413    58           3584      sigismember
47.08      0.205434    58           3520      sigaddset
2.32       0.010103    89           113       sigaction
0.74       0.003249    59           55       sigemptyset
0.47       0.002039    92           22       mmap
0.42       0.001822    455          4         pthread_create
0.36       0.001555    194          8         pthread_sigmask
0.14       0.000599    599          1         pthread_getattr_np
0.11       0.000476    95           5         malloc
0.11       0.000467    93           5         pthread_attr_init
0.10       0.000425    106          4         pthread_attr_getstacksize
0.10       0.000422    105          4         pthread_attr_setdetachstate
0.09       0.000404    101          4         sigfillset
0.04       0.000195    195          1         pthread_cond_broadcast
0.04       0.000187    187          1         pthread_mutex_unlock
0.04       0.000171    171          1         pthread_mutex_lock
0.03       0.000116    58           2         __errno_location
0.01       0.000063    63           1         free
0.01       0.000062    62           1         pthread_attr_getstack
0.01       0.000062    62           1         pthread_self
0.01       0.000060    60           1         pthread_attr_destroy
-----
100.00     0.436324                7338      total
amin@msi-pro: ~$
```

As shown above, the suspicious binary heavily used thread functions in addition to memory functions, indicating that the binary was intended to carry out memory and computationally heavy tasks. The next subsection will explain how `/proc` directory was used to analyze the suspicious process's behaviour.

/proc

The suspicious process was located under `/proc` directory as shown below:

```
root@msi-pro:/proc/47185# pwd
/proc/47185
root@msi-pro:/proc/47185#
```

The, the value of `/proc/pid/comm` file was “systemctl3.4”, and the value of `/proc/pid/cmdline` file was “victim” as shown below:

```
root@msi-pro:/proc/47185# cat comm
systemctl3.4
root@msi-pro:/proc/47185# cat cmdline
./victimroot@msi-pro:/proc/47185#
```

One of the tools had complained about the process named “systemctl3.4” earlier. And also the binary of the process is called “victim”, the very same binary that *Rkhunter* has complained about.

Then, the following command was executed to identify the current working directory of the malicious process:

```
root@msi-pro:/proc/47185# ls -l ./cwd
lrwxrwxrwx 1 root root 0 Jun  3 12:45 ./cwd -> /tmp
root@msi-pro:/proc/47185#
```

As shown above, the binary of the process is located under `/tmp` directory, a typical directory where malwares are stored. And the very same location where the suspicious binary had

been stored, proving with certainty that the suspicious binary is indeed the binary of the suspicious process.

Then, the following command was executed to list the open file descriptors of the process:

```
root@msi-pro:/proc/47185# ls -l fd
total 0
lrwx----- 1 root root 64 Jun  3 12:43 0 -> /dev/pts/1
lrwx----- 1 root root 64 Jun  3 12:43 1 -> /dev/pts/1
lrwx----- 1 root root 64 Jun  3 12:43 10 -> 'socket:[484822]'
lrwx----- 1 root root 64 Jun  3 12:43 11 -> 'socket:[483143]'
lrwx----- 1 root root 64 Jun  3 12:43 2 -> /dev/pts/1
lr-x----- 1 root root 64 Jun  3 12:43 3 -> /tmp/victim.json
lrwx----- 1 root root 64 Jun  3 12:43 4 -> 'anon_inode:[eventpoll]'
lrwx----- 1 root root 64 Jun  3 12:43 5 -> 'anon_inode:[eventfd]'
lrwx----- 1 root root 64 Jun  3 12:43 7 -> 'anon_inode:[eventfd]'
lrwx----- 1 root root 64 Jun  3 12:43 9 -> 'anon_inode:[eventfd]'
root@msi-pro:/proc/47185#
```

As shown above, the process has opened the file */tmp/victim.json*. The content of */tmp/victim.json* is as follows:

```
{
  "IfaceStr": "wlp164s0",
  "SrcMACStr": "a4:42:3b:4d:9d:2e",
  "DstMACStr": "c8:15:4e:d3:07:1b",
  "SrcIPStr": "10.0.0.2",
  "DstIPStr": "10.0.0.3",
  "SrcPortInt": 9999,
  "DstPortInt": 8888
}
```

As shown in the figure above, SrcIPStr is set to the IPv4 address of our machine, and DstIPStr is set to the IPv4 address of host 10.0.0.3. And SrcPortInt is set to 9999 and DstPortInt is set to 8888, the very same two ports that were generating a high number of half open TCP connections. Possibly, victim.json file was a config file that the malware developers were using to customize the malware for different machines on our network.

Then, the following command was executed to check ARP cache of the process:

```
root@msi-pro:/proc/47185/net# cat ./arp
IP address      HW type    Flags      HW address    Mask        Device
10.0.0.1        0x1       0x2       d4:e2:cb:4e:61:bd *            wlp34s0
10.0.0.3        0x1       0x2       a4:42:3b:4d:9d:2e *            wlp34s0
root@msi-pro:/proc/47185/net#
```

As shown above, there is a record of host 10.0.0.3.

Then, the following command was executed to check the TCP connections of the process:

```
root@msi-pro:/proc/47185/net# cat tcp
sl local_address rem_address  st tx_queue rx_queue tr tm->when retrnsmt uid timeout inode
0: 00000000:0016 00000000:0000 0A 00000000:00000000 00:00000000 00000000 0 0 31093 1 0000000041cb2168 100 0 0 10 0
1: 0100007F:0019 00000000:0000 0A 00000000:00000000 00:00000000 00000000 0 0 21145 1 00000000d2bea8c1 100 0 0 10 20
2: 0100007F:0277 00000000:0000 0A 00000000:00000000 00:00000000 00000000 0 0 93264 1 000000001fdf8b5a 100 0 0 10 0
3: 0200000A:C204 BE63FABE:01BB 01 00000000:00000000 02:000000BD 00000000 1000 0 556999 2 000000005cfb98af 25 4 28 10 -1
4: 0200000A:E0D2 0300000A:0016 01 00000000:00000000 02:00095840 00000000 1000 0 136224 2 00000000acd2482b 42 4 28 7 7
5: 0200000A:C1F4 BE63FABE:01BB 01 00000000:00000000 02:000000BC 00000000 1000 0 556998 2 0000000086993e80 62 4 31 10 -1
root@msi-pro:/proc/47185/net#
```

Then, the following command was executed to check UDP traffic of the process:

```
root@msi-pro:/proc/47185/net# cat udp
sl local_address rem_address  st tx_queue rx_queue tr tm->when retrnsmt uid timeout inode ref pointer drops
2204: 0200000A:0044 0100000A:0043 01 00000000:00000000 00:00000000 00000000 0 0 567836 2 0000000013d56170 0
3428: 00000000:850C 00000000:0000 07 00000000:00000000 00:00000000 00000000 104 0 14702 2 00000000926fdcc3 0
7489: FB0000E0:14E9 00000000:0000 07 00000000:00000000 00:00000000 00000000 1000 0 557012 2 00000000c45a55a7 0
7489: 00000000:14E9 00000000:0000 07 00000000:00000000 00:00000000 00000000 104 0 14700 2 000000009a8cfa7d 0
root@msi-pro:/proc/47185/net#
```

The next section enumerates the defensive measures we took to counter the malware.

Defence Actions

Upon discovering the malware's presence on msi-pro workstation, and analyzing the behavior of the malware, we moved on to take a set of defensive actions to eradicate the malware. First, we wrote some *iptables* rules to block all traffic to and from host 10.0.0.3. And then we wrote another set of *iptables* rules to detect port knocks from any host in case the malware developers changed their IP address to evade the first set of *iptables* rules we had written. Then we killed the malicious process using the kill command. Then all cron jobs were cleared in case the malware was using cron jobs to persist. Then we fully updated the msi-pro machine lest the malware developers were exploiting an outdated package. And then we set up *AppArmor*. The next subsection explains how we wrote the *iptables* rules to block all inbound and outbound traffic to host 10.0.0.3

iptables

The following commands were executed to block all traffic to and from host 10.0.0.3:

```
iptables -A INPUT -s 10.0.0.3 -j DROP

iptables -A OUTPUT -d 10.0.0.3 -j DROP

iptables -A FORWARD -s 10.0.0.3 -j DROP
iptables -A FORWARD -d 10.0.0.3 -j DROP

sudo apt update
sudo apt install iptables-persistent

sudo iptables-save > /etc/iptables/rules.v4
sudo ip6tables-save > /etc/iptables/rules.v6
```

Then the following commands were executed to detect the port knock and block any inbound and outbound traffic to the host performing the port knocking:

```
modprobe xt_recent

iptables -A INPUT -p tcp --dport 6000 --syn -m recent --name stage1 --set

iptables -A INPUT -p tcp --dport 7000 --syn \
-m recent --name stage1 --rcheck --seconds 5 --reap \
-m recent --name stage2 --set

iptables -A INPUT -p tcp --dport 8000 --syn \
-m recent --name stage2 --rcheck --seconds 5 --reap \
-m recent --name stage3 --set

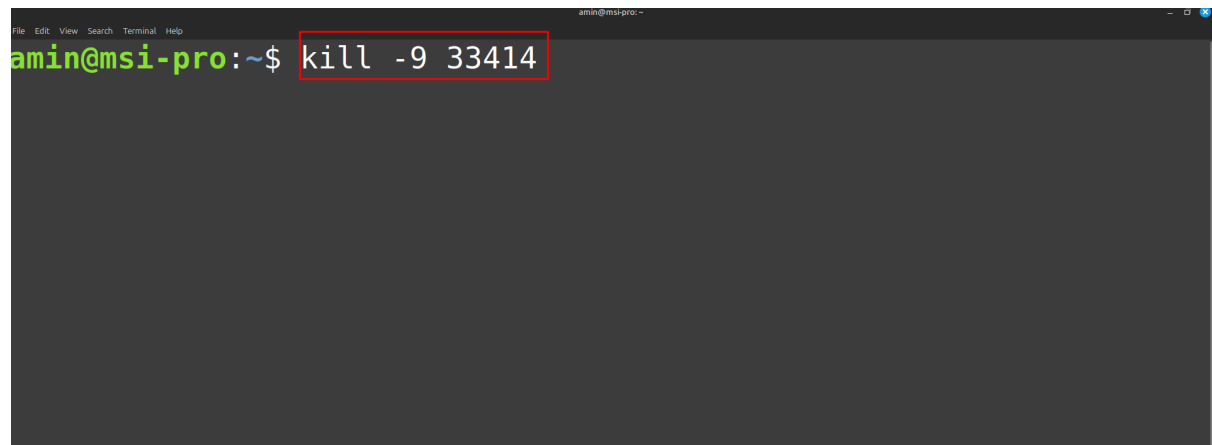
iptables -A INPUT -p tcp --dport 6000 --syn \
-m recent --name stage3 --rcheck --seconds 5 --reap \
-m recent --name blacklist --set
```

```
iptables -A INPUT -m recent --name blacklist --rcheck --seconds 1000000000000 --reap  
-j DROP
```

The next sub-section explains how the kill command was used to kill the malicious process.

Killing Malicious Process

The following command was executed to kill the parent malicious process along with all its child processes it had forked:

A terminal window titled 'amin@msi-pro' with a menu bar (File, Edit, View, Search, Terminal, Help). The prompt is 'amin@msi-pro:~\$'. The command 'kill -9 33414' is entered and highlighted with a red box. The terminal background is dark grey.

```
amin@msi-pro:~$ kill -9 33414
```

The next subsection explains how all *cron* jobs were cleared in case the malware was using *cron* jobs to persist reboots and process kills.

Clearing Cron Jobs

The following commands were executed to eradicate all *cron* jobs on the workstation:

```
crontab -r  
sudo crontab -r  
sudo systemctl stop cron
```

The next subsection explains how all packages on the system were updated lest the malware was exploiting vulnerabilities of updated packages.

Updating Packages

The following command was executed to upgrade all packages on msi-pro workstation:

```
amin@msi-pro:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for amin:
Hit:1 https://brave-browser-apt-release.s3.brave.com stable InRelease
Hit:2 https://dl.google.com/linux/chrome/deb stable InRelease
Ign:3 http://packages.linuxmint.com wilma InRelease
Hit:4 http://packages.linuxmint.com wilma Release
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:6 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:8 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:9 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... 98%
```

The next subsection explains how *AppArmor* was set up to protect msi-pro workstation against threats.

AppArmor

A shell script was written to set up AppArmor on the msi-pro workstation.

The following commands were executed to create and give executable permission to the script file:

```
amin@msi-pro:~$ touch install-apparmor.sh
amin@msi-pro:~$ chmod +x ./install-apparmor.sh
amin@msi-pro:~$ vim ./install-apparmor.sh
```

The following is a snapshot of the script used to set up AppArmor:

```
set -euo pipefail

REBOOT_NEEDED=0
GRUB_DROPIN="/etc/default/grub.d/apparmor.cfg"

need_root() {
    if [[ $EUID -ne 0 ]]; then
        echo "ERROR: This script must be run as root." >&2
        exit 1
    fi
}

check_debian() {
    if [[ -f /etc/debian_version ]]; then
        echo "[✓] Debian detected: $(cat /etc/debian_version)"
    else
        echo "ERROR: This script is intended for Debian systems only." >&2
        exit 1
    fi
}

kernel_has_apparmor() {
    grep -q "CONFIG_SECURITY_APPARMOR=y" "/boot/config-$(uname -r)" && \
    ". /install-apparmor.sh" 92L, 2394B written
}
```

18,5 Top

Then, the script was executed to set up AppArmor on the msi-pro workstation. The next section will explain how the artifacts of the malware were examined.

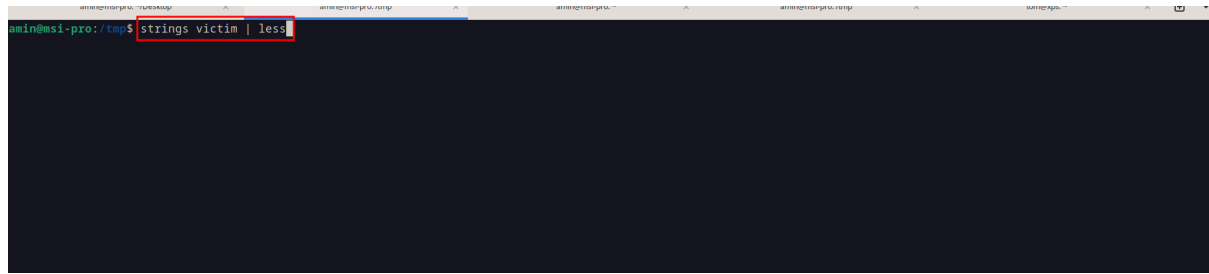
Artifacts

The artifacts of the malware were analyzed to understand with certainty how the malware operated. Specifically, the “victim” binary was reverse engineered using various tools to understand the inner workings of it. Initially, the *strings* command was used to list the readable strings encoded within the malicious binary. Most of the readable strings were function names with prefix *pcap*, indicating that the binary heavily used Linux’s *pcap* library for packet construction. Then, the malicious binary was uploaded to *VirusTotal* for analysis. *VirusTotal* reported that the binary modified various files including */etc/passwd* and */etc/shadow*, indicating that the malware was trying to exfiltrate our password. Then the *objdump* command was used to convert the binary into assembly. Then, the *file* command was used to identify the file type, which turned out to be ELF. And then the *readelf* command was used to list the file details. At this point, we were certain that the malware was developed using the Go programming language.

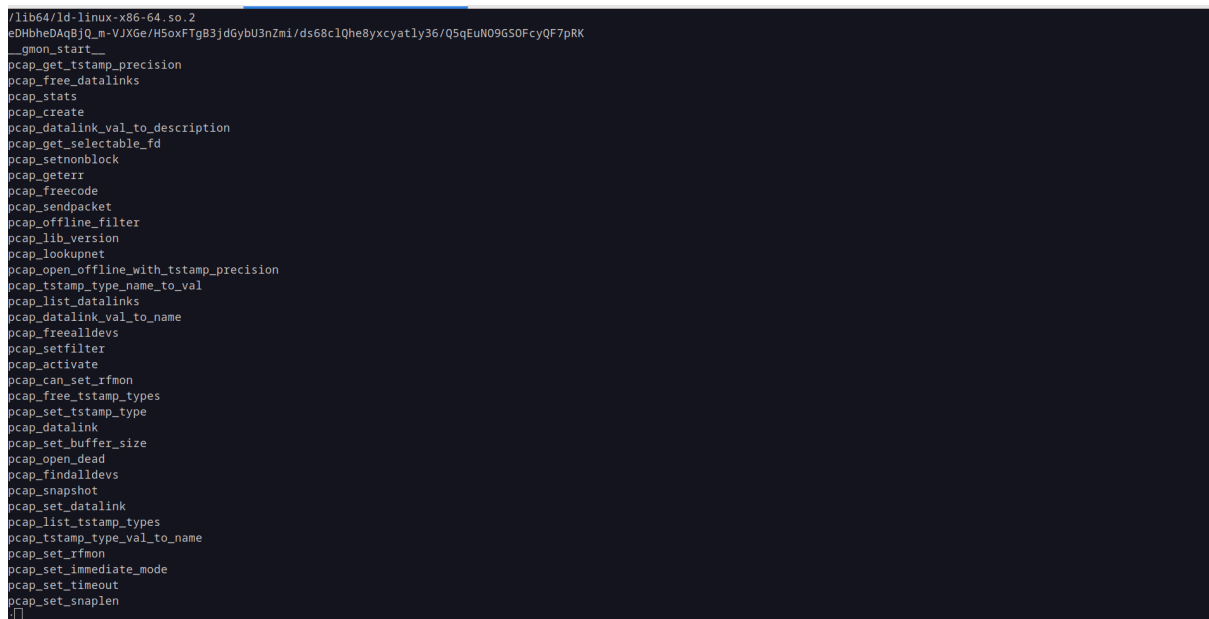
Moverover, *nm* command was used to list symbol names in the malicious binary file. We were able to recover numerous function and variable names, understanding how the malware worked just by looking at the function and variable names. For instance, numerous function and variable names had the prefix *c2/beacon*; we suspect that *c2* means command and control, so these function and variable names were responsible for implementing the command and control beacons. Additionally, some function and variable names had the prefix, *c2/util*, meaning command and control util library. Additionally, we were able to recover some *Github* repository links that the malware was using in addition to some standard *Golang* libraries, all of which helped us to understand the internal workings of the malware. The next subsection explains how the *strings* command was used to list the printable strings in the malicious binary file.

strings

The following command was executed to list printable strings in the suspicious binary:

A terminal window with a dark background. The prompt is 'admin@msi-pro: /tmp\$'. The command 'strings victim | less' is entered and highlighted with a red box. The terminal output is mostly black, indicating the 'less' command is in use to view the output of 'strings victim'.

The printable strings of the suspicious binary are as follows:

A terminal window showing the output of the 'strings' command. The output is a long list of printable strings, mostly function names from the pcap library, such as 'pcap_get_tstamp_precision', 'pcap_free_datalinks', 'pcap_stats', 'pcap_create', 'pcap_datalink_val_to_description', 'pcap_get_selectable_fd', 'pcap_setnonblock', 'pcap_geterr', 'pcap_freecode', 'pcap_sendpacket', 'pcap_offline_filter', 'pcap_lib_version', 'pcap_lookupnet', 'pcap_open_offline_with_tstamp_precision', 'pcap_tstamp_type_name_to_val', 'pcap_list_datalinks', 'pcap_datalink_val_to_name', 'pcap_freealldevs', 'pcap_setfilter', 'pcap_activate', 'pcap_can_set_rfmmon', 'pcap_free_tstamp_types', 'pcap_set_tstamp_type', 'pcap_datalink', 'pcap_set_buffer_size', 'pcap_open_dead', 'pcap_findalldevs', 'pcap_snapshot', 'pcap_set_datalink', 'pcap_list_tstamp_types', 'pcap_tstamp_type_val_to_name', 'pcap_set_rfmmon', 'pcap_set_immediate_mode', 'pcap_set_timeout', and 'pcap_set_snaplen'. The list ends with a newline character.

As shown in the figure above, the majority of the printable names are functions with *pcap* prefix belonging to the *pcap* library and used for packet construction. The next subsection explains how *VirtusTotal* was used.

VirusTotal

The suspicious binary was uploaded to [Virtustotal.com](https://www.virustotal.com) as shown below:

0
/ 65
Community Score

No security vendors flagged this file as malicious

Reanalyze Similar More

85c085ecf4a812796984dd117a0da79725cb456fc833be77dc4cbfe9243ef3c4

Size
6.29 MB

Last Analysis Date
a moment ago

AQ
ELF

DETECTION

DETAILS

BEHAVIOR

COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Security vendors' analysis ⓘ

Do you want to automate checks?

Acronis (Static ML)	✓ Undetected	AhnLab-V3	✓ Undetected
AliCloud	✓ Undetected	ALYac	✓ Undetected
Antiy-AVL	✓ Undetected	Arcabit	✓ Undetected
Avast	✓ Undetected	Avast-Mobile	✓ Undetected
AVG	✓ Undetected	Avira (no cloud)	✓ Undetected
Baidu	✓ Undetected	BitDefender	✓ Undetected

VirtusTotal marked the binary as suspicious as shown below:

Max size 650MB 85c085ecf4a812796984dd117a0da79725cb456fc833be77dc4cbfe9243ef3c4

There are some sandboxes still analysing the file.

Activity Summary

Download Artifacts Full Reports Help

⚠ Detections
NOT FOUND

🏷 Mitre Signatures
NOT FOUND

📋 IDS Rules
1 LOW

🔗 Sigma Rules
NOT FOUND

🗑 Dropped Files
NOT FOUND

🌐 Network comms
1 DNS 2 IP 1 JA3

🚨 Crowdsourced IDS rules ⓘ
Matches rule (tcp) experimental TCP options found at Snort registered user ruleset
protocol-command-decode

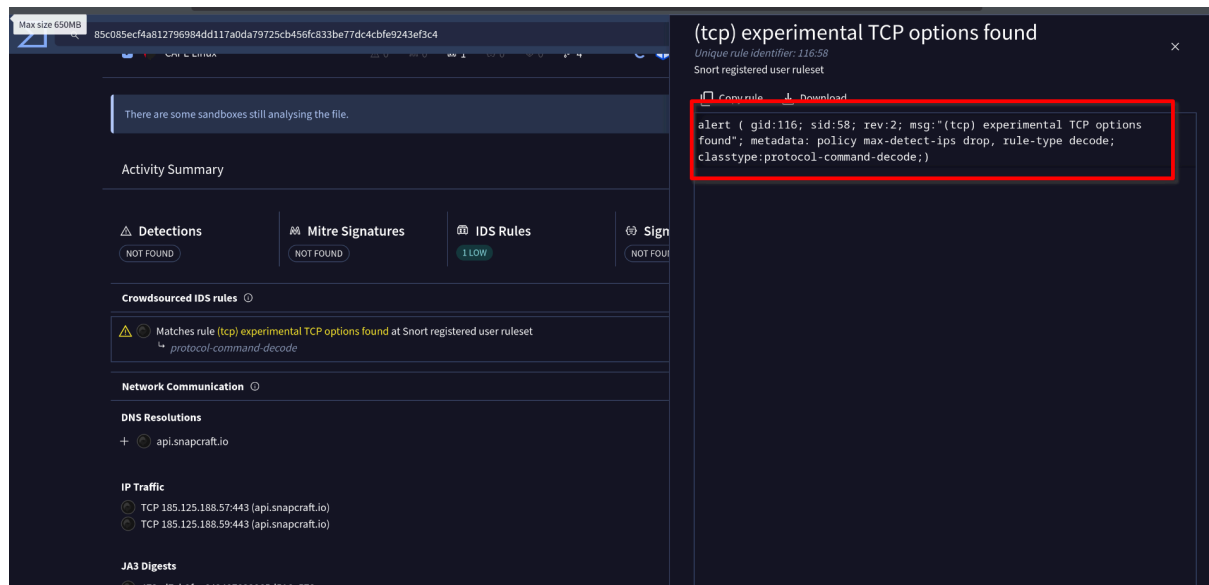
🌐 Network Communication ⓘ

DNS Resolutions
+ api.snapcraft.io

IP Traffic
TCP 185.125.188.57:443 (api.snapcraft.io)
TCP 185.125.188.59:443 (api.snapcraft.io)

JA3 Digests
473cd7cb9faa642487833865d516e578

The following figure shows the Snort rules that *VirtusTotal* used to detect suspicious network traffic generated by the binary:

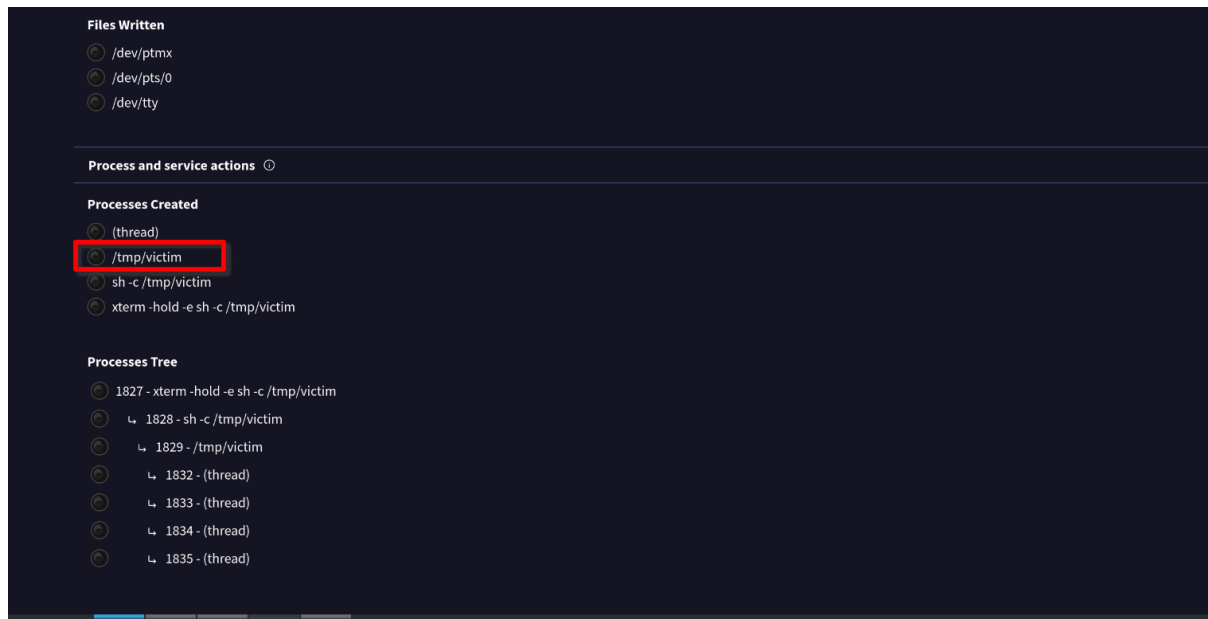


The following is a list of files that the binary had opened according to *VirtusTotal*:



As shown in the figure above, some critical files such as `/etc/passwd`, `/etc/group`, and `/etc/shells` were opened by the binary. This indicates that the malware was trying to set up a user account along a group in order to persist on the msi-pro machine.

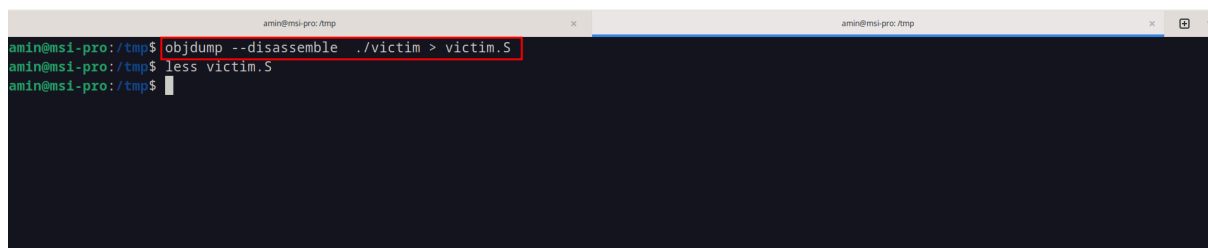
The following figure shows the exact location of where the malicious process stores its binary:



As shown in the figure above, the binary is stored in the `/tmp` directory, indicating that no matter where the binary is executed in the filesystem the malware process eventually moves the binary file to `/tmp` directory to evade detection. The next subsection explains how `objdump` was used.

objdump

The following command was executed to reverse engineer the malware, and paste the assembly code onto `victim.S` file as shown below:



The assembly code of the malware is as follows:

```
./victim:      file format elf64-x86-64

Disassembly of section .init:

000000000403000 <_init>:
403000:  48 83 ec 08          sub    $0x8,%rsp
403004:  48 8b 05 c5 2f 44 00  mov    0x442fc5(%rip),%rax      # 845fd0 <__gmon_start__@Base>
40300b:  48 85 c0             test   %rax,%rax
40300e:  74 02               je     403012 <_init+0x12>
403010:  ff d0              call   *%rax
403012:  48 83 c4 08          add    $0x8,%rsp
403016:  c3                 ret

Disassembly of section .plt:

000000000403020 <pcap_list_tstamp_types@plt-0x10>:
403020:  ff 35 ca 2f 44 00    push  0x442fca(%rip)          # 845ff0 <_GLOBAL_OFFSET_TABLE_+0x8>
403026:  ff 25 cc 2f 44 00    jmp    *0x442fcc(%rip)        # 845ff8 <_GLOBAL_OFFSET_TABLE_+0x10>
40302c:  0f 1f 40 00          nopl   0x0(%rax)

000000000403030 <pcap_list_tstamp_types@plt>:
403030:  ff 25 ca 2f 44 00    jmp    *0x442fca(%rip)        # 846000 <pcap_list_tstamp_types@Base>
403036:  68 00 00 00 00 00    push  $0x0
40303b:  e9 e0 ff ff          jmp    403020 <_init+0x20>

000000000403040 <pcap_set_promisc@plt>:
403040:  ff 25 c2 2f 44 00    jmp    *0x442fc2(%rip)        # 846008 <pcap_set_promisc@Base>
403046:  68 01 00 00 00 00    push  $0x1
40304b:  e9 d0 ff ff          jmp    403020 <_init+0x20>
victim.S
```

The next subsection explains how the *file* command was used to identify the exact file type of the malicious binary file.

file command

The following command was used to identify the exact file type of the binary:

```
amin@msi-pro:/tmp$ pwd
/tmp
amin@msi-pro:/tmp$ file victim
victim: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f8835a9f290cea29d9753906887399b5f42a367b, for GNU/Linux 3.2.0, with debug_info, not stripped
amin@msi-pro:/tmp$
```

As shown above, the binary file is of type ELF 64-bit. The next subsection explains how *readelf* command was used.

readelf command

The following command to display all information about the *ELF* file:

```
amin@msi-pro:/tmp$ readelf --all victim
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  EXEC (Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x403690
  Start of program headers:              64 (bytes into file)
  Start of section headers:              6590680 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:             14
  Size of section headers:               64 (bytes)
  Number of section headers:             51
  Section header string table index: 50

Section Headers:
 [Nr] Name              Type              Address             Offset
     Size              EntSize          Flags Link Info  Align
 [ 0] 0000000000000000 NULL              0000000000000000  0 0 0
 [ 1] .interp              PROGBITS          0000000000400350  00000350
     000000000000001c 0000000000000000  A  0  0  1
 [ 2] .note.gnu.pr[...]    NOTE              0000000000400370  00000370
     0000000000000020 0000000000000000  A  0  0  8
 [ 3] .note.gnu.bu[...]    NOTE              0000000000400390  00000390
     0000000000000024 0000000000000000  A  0  0  4
 [ 4] .note.ABI-tag        NOTE              00000000004003b4  000003b4
     0000000000000020 0000000000000000  A  0  0  4
 [ 5] .note.go.buildid     NOTE              00000000004003d4  000003d4
     0000000000000010 0000000000000000  A  0  0  1
```

As shown below, names of functions and modules used to develop the malware are discovered by analyzing the symbol section of the *ELF* file:

```
File Edit View Search Terminal Help
1863: 0000000000493b20 175 FUNC LOCAL DEFAULT 15 time.NewTimer
1864: 0000000000493be0 105 FUNC LOCAL DEFAULT 15 time.(*Timer).Reset
1865: 0000000000493c60 173 FUNC LOCAL DEFAULT 15 time.sendTime
1866: 0000000000493d20 86 FUNC LOCAL DEFAULT 15 time.AfterFunc
1867: 0000000000493d80 117 FUNC LOCAL DEFAULT 15 time.goFunc
1868: 0000000000493e00 92 FUNC LOCAL DEFAULT 15 time.open
1869: 0000000000493e60 44 FUNC LOCAL DEFAULT 15 time.closefd
1870: 0000000000493ea0 339 FUNC LOCAL DEFAULT 15 time.preadn
1871: 0000000000494000 208 FUNC LOCAL DEFAULT 15 time.NewTicker
1872: 00000000004940e0 54 FUNC LOCAL DEFAULT 15 time.(*Ticker).Stop
1873: 0000000000494120 156 FUNC LOCAL DEFAULT 15 time.(*Time).addSec
1874: 00000000004941c0 74 FUNC LOCAL DEFAULT 15 time.Time.IsZero
1875: 0000000000494220 179 FUNC LOCAL DEFAULT 15 time.Time.After
1876: 00000000004942e0 179 FUNC LOCAL DEFAULT 15 time.Time.Before
1877: 00000000004943a0 170 FUNC LOCAL DEFAULT 15 time.Time.Equal
1878: 0000000000494460 127 FUNC LOCAL DEFAULT 15 time.dateToAbsDays
1879: 00000000004944e0 184 FUNC LOCAL DEFAULT 15 time.absDays.date
1880: 00000000004945a0 195 FUNC LOCAL DEFAULT 15 time.absDays.yearYday
1881: 0000000000494680 262 FUNC LOCAL DEFAULT 15 time.Time.absSec
1882: 00000000004947a0 295 FUNC LOCAL DEFAULT 15 time.Time.locabs
1883: 00000000004948e0 109 FUNC LOCAL DEFAULT 15 time.Duration.String
1884: 0000000000494960 1206 FUNC LOCAL DEFAULT 15 time.Duration.format
1885: 0000000000494e20 357 FUNC LOCAL DEFAULT 15 time.Time.Add
1886: 0000000000494fa0 444 FUNC LOCAL DEFAULT 15 time.Time.Sub
1887: 0000000000495160 201 FUNC LOCAL DEFAULT 15 time.Until
1888: 0000000000495240 121 FUNC LOCAL DEFAULT 15 time.Time.Zone
1889: 00000000004952c0 766 FUNC LOCAL DEFAULT 15 time.Time.Append[...]
1890: 00000000004955c0 153 FUNC LOCAL DEFAULT 15 time.Time.Marshal[...]
1891: 0000000000495660 778 FUNC LOCAL DEFAULT 15 time.(*Time).Unm[...]
1892: 0000000000495980 326 FUNC LOCAL DEFAULT 15 time.Time.MarshalJSON
1893: 0000000000495ae0 244 FUNC LOCAL DEFAULT 15 time.(*Time).Unm[...]
1894: 0000000000495be0 278 FUNC LOCAL DEFAULT 15 time.Time.appendTo
1895: 0000000000495d00 147 FUNC LOCAL DEFAULT 15 time.Time.MarshalText
1896: 0000000000495da0 147 FUNC LOCAL DEFAULT 15 time.(*Time).Unm[...]
1897: 0000000000495e40 978 FUNC LOCAL DEFAULT 15 time.Date
1898: 0000000000496220 110 FUNC LOCAL DEFAULT 15 time.(*Location).get
1899: 0000000000496220 53 FUNC LOCAL DEFAULT 15 time.(*Location).get
```

The next subsection explains how the *nm* command was used to recover the symbols embedded in the malicious binary file.

nm

The following command was executed to list all global (extern) functions in the binary:

```
amin@msi-pro: $ nm --extern-only /tmp/victim
U abort@GLIBC_2.2.5
00000000085a0a1 B __bss_start
U calloc@GLIBC_2.2.5
0000000005d6c40 T _cgo_2b4b51eac36b_Cfunc__Cmalloc
0000000005d6d40 T _cgo_2b4b51eac36b_Cfunc_free
0000000005d6d50 T _cgo_2b4b51eac36b_Cfunc_setProcName
0000000005d6d60 T _cgo_2b4b51eac36b_Cfunc_spoofCmdline
0000000005d6e70 T _cgo_77133bf98b3a_C2func_getaddrinfo
0000000005d6db0 T _cgo_77133bf98b3a_C2func_getnameinfo
0000000005d6d70 T _cgo_77133bf98b3a_Cfunc__Cmalloc
0000000005d6ed0 T _cgo_77133bf98b3a_Cfunc_free
0000000005d6ee0 T _cgo_77133bf98b3a_Cfunc_freeaddrinfo
0000000005d6ef0 T _cgo_77133bf98b3a_Cfunc_gai_strerror
0000000005d6f20 T _cgo_77133bf98b3a_Cfunc_getaddrinfo
0000000005d6e20 T _cgo_77133bf98b3a_Cfunc_getnameinfo
0000000005d6f60 T _cgo_77133bf98b3a_Cfunc_res_search
0000000005d7070 T _cgo_c9e2e4b8ffce_Cfunc_calloc
0000000005d6fa0 T _cgo_c9e2e4b8ffce_Cfunc__Cmalloc
0000000005d70a0 T _cgo_c9e2e4b8ffce_Cfunc_fdopen
0000000005d70d0 T _cgo_c9e2e4b8ffce_Cfunc_free
0000000005d70e0 T _cgo_c9e2e4b8ffce_Cfunc_pcap_activate
0000000005d7110 T _cgo_c9e2e4b8ffce_Cfunc_pcap_can_set_rfmon
0000000005d7140 T _cgo_c9e2e4b8ffce_Cfunc_pcap_close
```

As shown in the figure above, functions with suffix *setProcName* and *spoofCmdline* were most likely used to change the name of the malicious process and make it look innocuous. Also functions with suffix *getaddrinfo* and *getnameinfo* were most likely used to facilitate networking.

Additionally, the following symbols excerpted from *nm* output seem to indicate *pcap* library, process name spoofing, multi-threading, and privilege escalation:

```
_cgo_2b4b51eac36b_Cfunc_setProcName
_cgo_2b4b51eac36b_Cfunc_spoofCmdline
_cgo_77133bf98b3a_C2func_getaddrinfo
_cgo_77133bf98b3a_C2func_getnameinfo
_cgo_c9e2e4b8ffce_Cfunc_pcap_dataink
_cgo_c9e2e4b8ffce_Cfunc_pcap_open_live
_cgo_c9e2e4b8ffce_Cfunc_pcap_open_dead
_cgo_c9e2e4b8ffce_Cfunc_pcap_sendpacket
_cgo_c9e2e4b8ffce_Cfunc_pcap_setfilter
_cgo_c9e2e4b8ffce_Cfunc_pcap_set_promisc
_cgo_libc_setegid
_cgo_libc_seteuid
_cgo_sys_thread_start
x_cgo_notify_runtime_init_done
```

```
amin@msi-pro:~$ nm /tmp/victim --debug-syms
0000000000000000 a
000000000006b14f4 r $f32.358637bd
000000000006b1508 r $f32.3f000000
000000000006b14fc r $f32.4b189680
000000000006b14f8 r $f32.6258d727
000000000006b1500 r $f32.80000000
000000000006b1504 r $f32.cb189680
000000000006b16a8 r $f64.3eb0000000000000
000000000006b1550 r $f64.3eb0c6f7a0b5ed8d
000000000006b16b0 r $f64.3f047aa147aa147b
```

```
0000000000004d8680 t bytes.TrimSpace
00000000000085a110 b c2/beacon.border
00000000000085a0e0 b c2/beacon.CodeStrToCodeNum
00000000000085a0f8 b c2/beacon.errColor
00000000000085a0f0 b c2/beacon.info
00000000000058d180 t c2/beacon.init
00000000000084e000 d c2/beacon.inittask
00000000000058d6a0 t c2/beacon.MakeStenoFieldsRSTACK
00000000000058d360 t c2/beacon.map.init.0
00000000000085a100 b c2/beacon.optionName
00000000000085a108 b c2/beacon.optionNum
00000000000058e180 t c2/beacon.parseRoute
00000000000058d7c0 t c2/beacon.parseStenoFieldsSYN
000000000000847430 d c2/beacon.PortSequence
00000000000085a0e8 b c2/beacon.prompt
00000000000058f560 t c2/beacon.randPort
00000000000058d920 t c2/beacon.ReceiveKnock
00000000000058e120 t c2/beacon.ReceiveKnock.deferwrap1
00000000000058e700 t c2/beacon.receiver
```

The following is a portion of the output of *nm* that explain a lot about the malware:

53

```

000000000058f340 t c2/beacon.transmitStenoFields
000000000058fd60 t c2/beacon.Victim
000000000058f6c0 t c2/beacon.VictimSetup0
0000000000846348 d c2/util._cgo_2b4b51eac36b_Cfunc_setProcName
0000000000846350 d c2/util._cgo_2b4b51eac36b_Cfunc_spoofCmdline
00000000005bd820 t c2/util.commanderRemoteShell
00000000005be180 t c2/util.ConnectTCP
00000000006b2700 r c2/util..dict.NetRead[c2/util.FileHeader]
00000000006b26e0 r c2/util..dict.NetRead[string]
00000000006b18e0 r c2/util..dict.NetWrite[c2/util.EventHeader]
00000000006b18f0 r c2/util..dict.NetWrite[c2/util.FileHeader]
00000000005bbe00 t c2/util.FileHash
00000000005bc060 t c2/util.FileHash.deferwrap1
00000000005bc760 t c2/util.isKeyboard
00000000005bc940 t c2/util.kbdLog
00000000005bcc00 t c2/util.Keylogger
000000000085a118 b c2/util.keyNames
00000000005c0400 t c2/util.NetRead[go.shape.string]
00000000005beb20 t c2/util.NetRead[go.shape.struct { Name string
"json:\"name\""; Size int64 "json:\"size\""}]]
00000000005be960 t c2/util.NetWrite[go.shape.struct { Event
github.com/fsnotify/fsnotify.Event "json:\"event\""; Size int64
"json:\"size\""; IsDirectory bool "json:isdirectory"}]]
00000000005bee00 t c2/util.NetWrite[go.shape.struct { Name string
"json:\"name\""; Size int64 "json:\"size\""}]]
00000000005bb380 t c2/util.RecvFile
00000000005bb600 t c2/util.RecvFile.deferwrap1
00000000005bd720 t c2/util.RemoteShell
00000000005bb160 t c2/util.SendFile
00000000005be840 t c2/util.SpoofPsName
00000000005bb660 t c2/util.WatchInodeVictim
00000000005bc0c0 t c2/util.WatchShadowFile

```

The following table explains what each of the symbol names shown above are likely to represent:

Symbol Name	Symbol Name Speculation
<i>c2/util.WatchShadowFile</i>	Could be a function used to watch <i>/etc/shadow</i> file.
<i>c2/util.WatchInodeVictim</i>	Could be a function used to watch an inode on our machine.
<i>c2/util.SpoofPsName</i>	Could be a function used to spoof process

	names.
<i>c2/util.SendFile</i>	Could be a function used to send a file from our device to host 10.0.0.3.
<i>c2/util.RemoteShell</i>	Could be a function used to start a remote shell to our machine.
<i>c2/util.RecvFile</i>	Could be used to receive a file from host 10.0.0.3 to our machine.
<i>c2/util.NetWrite[go.shape.struct { Name string "json:\\"name\\""; Size int64 "json:\\"size\\""}]</i>	It seems that JSON is being used to send files over the network.
<i>c2/util.NetWrite[go.shape.struct { Event github.com/fsnotify/fsnotify.Event "json:\\"event\\""; Size int64 "json:\\"size\\""; IsDirectory bool "json.isdirectory"}]</i>	It seems that <i>github.com/fsnotify/fsnotify</i> repository is being used for file and directory watching.
<i>c2/util.keyNames</i>	Could be a function used as part of a keylogger converting keystrokes to their corresponding string representation.
<i>c2/util.Keylogger</i>	Could be a function used to keylog.
<i>c2/util.kbdLog</i>	Could be a function used to keylog.
<i>c2/util.isKeyboard</i>	Could be a function used to check whether a device is a keyboard or not.
<i>c2/util.FileHash</i>	Could be a function used to compute file hash.
<i>c2/util.ConnectTCP</i>	Could be a function used to establish TCP connection.
<i>c2/util.commanderRemoteShell</i>	Could be a function used to connect our machine to host 10.0.0.3 via a remote shell.
<i>c2/beacon.ReceiveKnock</i>	Could be a function used to receive port knock from host 10.0.0.3.
<i>c2/beacon.PortSequence</i>	Could be a function or a data structure representing the port knock sequence to be received.
<i>c2/beacon.parseRoute</i>	Could be a function used to parse network routes.

The following command is used to demangle symbols and make them readable:

```
amin@msi-pro:~$ nm /tmp/victim --demangle
```

```
00000000006b14f4 r $f32.358637bd
00000000006b1508 r $f32.3f000000
00000000006b14fc r $f32.4b189680
00000000006b14f8 r $f32.6258d727
00000000006b1500 r $f32.80000000
00000000006b1504 r $f32.cb189680
00000000006b16a8 r $f64.3eb0000000000000
00000000006b1550 r $f64.3eb0c6f7a0b5ed8d
00000000006b16b0 r $f64.3f847ae147ae147b
00000000006b1690 r $f64.3fd0000000000000
00000000006b16b8 r $f64.3fd3333333333333
00000000006b1668 r $f64.3fe0000000000000
00000000006b16c0 r $f64.3fe8000000000000
00000000006b16c8 r $f64.3fee666666666666
00000000006b15e8 r $f64.3ff0000000000000
00000000006b16d0 r $f64.3ff199999999999a
00000000006b16d8 r $f64.3ff3333333333333
00000000006b1698 r $f64.4010000000000000
```

“crypto” seems to prefix numerous symbols as shown below:

```
00000000005d2260 t crypto/internal/fips140/aes.(*Block).BlockSize
00000000005d3660 t crypto/internal/fips140/aes.(*CBCDecrypter).CryptBlocks
00000000005d32c0 t crypto/internal/fips140/aes.(*CBCEncrypter).CryptBlocks
00000000005d3800 t crypto/internal/fips140/aes.cryptBlocksDecGeneric
00000000005d3460 t crypto/internal/fips140/aes.cryptBlocksEncGeneric
00000000005d42c0 t crypto/internal/fips140/aes.ctrBlocks
00000000005d4540 t crypto/internal/fips140/aes.ctrBlocks1
00000000005d5000 t crypto/internal/fips140/aes.ctrBlocks1Asm.abi0
00000000005d4600 t crypto/internal/fips140/aes.ctrBlocks2
00000000005d50e0 t crypto/internal/fips140/aes.ctrBlocks2Asm.abi0
00000000005d46c0 t crypto/internal/fips140/aes.ctrBlocks4
00000000005d5240 t crypto/internal/fips140/aes.ctrBlocks4Asm.abi0
00000000005d4780 t crypto/internal/fips140/aes.ctrBlocks8
00000000005d5480 t crypto/internal/fips140/aes.ctrBlocks8Asm.abi0
00000000005d3aa0 t crypto/internal/fips140/aes.(*CTR).XORKeyStream
00000000005d3b80 t crypto/internal/fips140/aes.(*CTR).XORKeyStreamAt
00000000005d2480 t crypto/internal/fips140/aes.decryptBlock
00000000005d4d60 t crypto/internal/fips140/aes.decryptBlockAsm.abi0
00000000005d2a00 t crypto/internal/fips140/aes.decryptBlockGeneric
```

Also, “encoding/base64” also prefixes numerous symbols as shown below:

```
0000000000591200 t encoding/base64.(*Encoding).AppendEncode
0000000000591900 t encoding/base64.(*Encoding).Decode
00000000005914e0 t encoding/base64.(*Encoding).decodeQuantum
0000000000590f60 t encoding/base64.(*Encoding).Encode
0000000000590c60 t encoding/base64.init
000000000084e100 d encoding/base64.inittask
000000000085a248 b encoding/base64.RawStdEncoding
000000000085a250 b encoding/base64.RawURLEncoding
000000000085a238 b encoding/base64.StdEncoding
00000000006b2020 r encoding/base64.stmp_1
00000000006b2030 r encoding/base64.stmp_2
000000000085a240 b encoding/base64.URLEncoding
00000000004de8e0 t encoding/binary.dataSize
00000000004dcd40 t encoding/binary.decodeFast
00000000006babe0 r encoding/binary.decodeFast.jump6
00000000004dee40 t encoding/binary.(*decoder).int16
00000000004deee0 t encoding/binary.(*decoder).int32
00000000004def80 t encoding/binary.(*decoder).int64
00000000004df020 t encoding/binary.(*decoder).value
```

Also, “encoding/json” also prefixes numerous symbols as shown below:

```
0000000000592be0 t encoding/json.(*decodeState).rescanLiteral
0000000000592a40 t encoding/json.(*decodeState).scanNext
0000000000592ae0 t encoding/json.(*decodeState).scanWhile
0000000000592980 t encoding/json.(*decodeState).skip
00000000005925a0 t encoding/json.(*decodeState).unmarshal
0000000000592d60 t encoding/json.(*decodeState).value
0000000000597b60 t encoding/json.(*decodeState).valueInterface
0000000000592f00 t encoding/json.(*decodeState).valueQuoted
00000000006b1a90 r encoding/json.dict.appendString[string]
00000000006b1aa0 r encoding/json.dict.appendString[[]uint8]
000000000059c260 t encoding/json.encodeByteSlice
000000000085ac40 b encoding/json.encoderCache
00000000005ad880 t encoding/json.(*encodeState).Bytes
00000000005ad900 t encoding/json.(*encodeState).Len
0000000000598880 t encoding/json.(*encodeState).marshal
00000000005989c0 t encoding/json.(*encodeState).marshal.func1
000000000085abc0 b encoding/json.encodeStatePool
00000000005ad940 t encoding/json.(*encodeState).Read
00000000005ada80 t encoding/json.(*encodeState).ReadByte
```

Also, “github.com/creack/pty” also prefixes numerous symbols as shown below:

```
00000000005bab60 t github.com/creack/pty.ptsname
00000000005bac00 t github.com/creack/pty.StartWithAttrs
00000000005baf40 t github.com/creack/pty.StartWithAttrs.func1
00000000005baf80 t github.com/creack/pty.StartWithSize
000000000085a198 b github.com/fatih/color.colorsCache
00000000008468c0 d github.com/fatih/color.Error
00000000004e32c0 t github.com/fatih/color.init
000000000084e080 d github.com/fatih/color..inittask
00000000004e3160 t github.com/fatih/color.New
0000000000da4e0c b github.com/fatih/color.NoColor
00000000008468b0 d github.com/fatih/color.Output
00000000006b1518 r github.com/fatih/color..stmp_1
0000000000da4e0d b github.com/fsnotify/fsnotify.debug
0000000000da53a0 b github.com/fsnotify/fsnotify.defaultBufferSize
000000000084e0c0 d github.com/fsnotify/fsnotify.defaultOpts
0000000000da4e0e b github.com/fsnotify/fsnotify.enableRecurse
0000000000846c80 d github.com/fsnotify/fsnotify.ErrClosed
0000000000846c90 d github.com/fsnotify/fsnotify.ErrEventOverflow
0000000000846c70 d github.com/fsnotify/fsnotify.ErrNonExistentWatch
```

Also, “github.com/fsnotify/fsnotify” also prefixes numerous symbols as shown below:

```
00000000005b3500 t github.com/fsnotify/fsnotify.(*Watcher).Close
00000000005b0560 t github.com/fsnotify/fsnotify.(*watches).removePath
00000000005b0980 t github.com/fsnotify/fsnotify.(*watches).updatePath
00000000005176e0 t github.com/google/gopacket.(*DecodeFailure).Error
0000000000517700 t github.com/google/gopacket.(*DecodeFailure).LayerContents
0000000000517720 t github.com/google/gopacket.(*DecodeFailure).LayerPayload
00000000005177a0 t github.com/google/gopacket.(*DecodeFailure).LayerType
0000000000517740 t github.com/google/gopacket.(*DecodeFailure).String
00000000005174e0 t github.com/google/gopacket.decodeFragment
0000000000846940 d github.com/google/gopacket.DecodeFragment
000000000051b020 t github.com/google/gopacket.(*DecodeFunc).Decode
0000000000517660 t github.com/google/gopacket.DecodeFunc.Decode
0000000000517140 t github.com/google/gopacket.decodePayload
0000000000846920 d github.com/google/gopacket.DecodePayload
000000000085a1a8 b github.com/google/gopacket.DecodersByLayerName
00000000005177c0 t github.com/google/gopacket.decodeUnknown
0000000000846930 d github.com/google/gopacket.DecodeUnknown
0000000000da4e58 b github.com/google/gopacket.Default
000000000051b3c0 t github.com/google/gopacket(*eagerPacket).AddLayer
```

Also, “golang.org/x/sys/unix” also prefixes numerous symbols as shown below:

```
00000000004e2bc0 t golang.org/x/sys/unix.InotifyInit1
00000000004e2ca0 t golang.org/x/sys/unix.InotifyRmWatch
00000000004e2960 t golang.org/x/sys/unix.ioctlPtr
00000000004e2f80 t golang.org/x/sys/unix.mmap
00000000004e2e60 t golang.org/x/sys/unix.mremap
00000000004e2d80 t golang.org/x/sys/unix.munmap
00000000004e30e0 t golang.org/x/sys/unix.RawSyscall.abi0
000000000084df88 d golang.org/x/sys/unix..stmp_0
000000000084df90 d golang.org/x/sys/unix..stmp_1
000000000084df98 d golang.org/x/sys/unix..stmp_2
0000000000849060 d golang.org/x/sys/unix..stmp_3
00000000006b1538 r golang.org/x/sys/unix..stmp_68
00000000004e30c0 t golang.org/x/sys/unix.Syscall6.abi0
00000000004e30a0 t golang.org/x/sys/unix.Syscall.abi0
```

Also, “hash/crc32” also prefixes numerous symbols as shown below:

```
000000000051c6a0 t hash/crc32.init.func2
000000000051c780 t hash/crc32.init.func2.1
000000000051c3c0 t hash/crc32.init.OnceFunc.func3
000000000051c480 t hash/crc32.init.OnceFunc.func3.1
000000000051c360 t hash/crc32.init.OnceFunc.func4
000000000051c220 t hash/crc32.init.OnceFunc.func5
000000000051c2e0 t hash/crc32.init.OnceFunc.func5.1
000000000051c1c0 t hash/crc32.init.OnceFunc.func6
000000000084e1c0 d hash/crc32..inittask
000000000051c7e0 t hash/crc32.MakeTable
000000000051d140 t hash/crc32.slicingMakeTable
000000000051d200 t hash/crc32.slicingUpdate
00000000006b2640 r hash/crc32..stmp_0
00000000006b2650 r hash/crc32..stmp_1
000000000051c8c0 t hash/crc32.update
000000000085a3c8 b hash/crc32.updateCastagnoli
000000000085a3e8 b hash/crc32.updateIEEE
0000000000405c20 t indexbody
0000000000405fc0 t indexbytebody
```

Also, “os.exec” also prefixes numerous symbols as shown below:

```
0000000000846d70 d os/exec..stmp_0
0000000000848820 d os/exec..stmp_1
0000000000848860 d os/exec..stmp_2
0000000000846d80 d os/exec..stmp_3
0000000000846d90 d os/exec..stmp_4
00000000004c68e0 t os.executable
00000000005ba5a0 t os/exec.(*wrappedError).Error
00000000005b7140 t os/exec.wrappedError.Error
00000000005ba640 t os/exec.(*wrappedError).Unwrap
00000000005b71e0 t os/exec.wrappedError.Unwrap
00000000004c9a00 t os.Exit
00000000004c7f60 t os.(*file).close
00000000004c7800 t os.(*File).Close
00000000004cac00 t os.(*File).copyFileRange
00000000004c69a0 t os.(*File).Name
00000000004c69c0 t os.(*File).Read
00000000004c4860 t os.(*File).readdir
00000000004c4580 t os.(*File).ReadDir
00000000004c5560 t os.(*File).readdir.deferwrap1
```

Also, “os” also prefixes numerous symbols as shown below:

```
00000000004c9b60 t os.Lstat
00000000004ca020 t os.lstatNolog
00000000004c79c0 t os.newFile
00000000004c7900 t os.NewFile
00000000004c5a80 t os.newHandleProcess
00000000004c8500 t os.newUnixDirent
00000000004cb3e0 t os.(*noReadFrom).ReadFrom
00000000004c6b60 t os.noReadFrom.ReadFrom
00000000004cbb80 t os.(*noWriteTo).WriteTo
00000000004c6ea0 t os.noWriteTo.WriteTo
00000000004c7780 t os.open
00000000004c7080 t os.openDir
00000000004c7de0 t os.openDirNolog
00000000004c6fe0 t os.OpenFile
00000000004c7be0 t os.openFileNolog
00000000004c98a0 t os.Pipe
0000000000846368 d os.pollCopyFileRange
0000000000846370 d os.pollSplice
00000000004c5c20 t os.(*Process).handlePersistentRelease
```

Also, “path/filepath” also prefixes numerous symbols as shown below:

```
00000000005ae460 t path/filepath.init
000000000084e060 d path/filepath..inittask
00000000005afaa0 t path/filepath.join
00000000005ae500 t path/filepath.Match
00000000005ae7c0 t path/filepath.matchChunk
00000000005ae6e0 t path/filepath.scanChunk
000000000085a520 b path/filepath.SkipAll
000000000085a510 b path/filepath.SkipDir
0000000000846880 d path/filepath..stmp_0
00000000005af580 t path/filepath.unixAbs
00000000005af640 t path/filepath.walkDir
00000000005af960 t path/filepath.WalkDir
00000000005afb40 t path/filepath.walkSvmlinks
```

Also, “vendor/golang.org/x/net/dns/dnsmessage” also prefixes numerous symbols as shown below:

```
00000000004ecd80 t vendor/golang.org/x/net/dns/dnsmessage.skipResource
0000000000847130 d vendor/golang.org/x/net/dns/dnsmessage..stmp_0
0000000000847140 d vendor/golang.org/x/net/dns/dnsmessage..stmp_1
00000000008471c0 d vendor/golang.org/x/net/dns/dnsmessage..stmp_10
00000000008471d0 d vendor/golang.org/x/net/dns/dnsmessage..stmp_11
00000000008471e0 d vendor/golang.org/x/net/dns/dnsmessage..stmp_12
00000000008471f0 d vendor/golang.org/x/net/dns/dnsmessage..stmp_13
0000000000847200 d vendor/golang.org/x/net/dns/dnsmessage..stmp_14
0000000000847210 d vendor/golang.org/x/net/dns/dnsmessage..stmp_15
0000000000847220 d vendor/golang.org/x/net/dns/dnsmessage..stmp_16
0000000000847230 d vendor/golang.org/x/net/dns/dnsmessage..stmp_17
0000000000847240 d vendor/golang.org/x/net/dns/dnsmessage..stmp_18
0000000000847150 d vendor/golang.org/x/net/dns/dnsmessage..stmp_2
0000000000847160 d vendor/golang.org/x/net/dns/dnsmessage..stmp_3
0000000000847170 d vendor/golang.org/x/net/dns/dnsmessage..stmp_4
0000000000847180 d vendor/golang.org/x/net/dns/dnsmessage..stmp_5
0000000000847190 d vendor/golang.org/x/net/dns/dnsmessage..stmp_6
00000000008471a0 d vendor/golang.org/x/net/dns/dnsmessage..stmp_7
00000000008471b0 d vendor/golang.org/x/net/dns/dnsmessage..stmp_9
```

The following table explains the meaning of each of the aforementioned prefixes gathered upon demangling the symbols of the binary files:

Prefix	Prefix Explanation	Malware Use Case speculation
<i>crypto</i>	crypto is a package of Go lang used for encryption. https://pkg.go.dev/crypto	Used to encrypt the content of inbound and outbound packet payloads.
<i>encoding/base64</i>	Base64 is a group of binary-to-text encoding schemes that transform binary data into a sequence of printable characters, limited to a set of 64 unique characters. https://pkg.go.dev/encoding/base64	Used to text parsing.

<i>encoding/json</i>	Used to convert to JSON and parse JSON. https://pkg.go.dev/encoding/json	Used for data exfiltration.
<i>https://pkg.go.dev/github.com/creack/pty</i>	Pty is a Go package for using unix pseudo-terminals.	Used for remote shell.
<i>github.com/fsnotify/fsnotify</i>	fsnotify is a Go library to provide cross-platform filesystem notifications on Windows, Linux, macOS, BSD, and illumos.	Used for file and directory watching,
<i>golang.org/x/sys/unix</i>	The sys/unix package provides access to the raw system call interface of the underlying operating system.	Used to execute system calls from Go Lang
<i>hash/crc32</i>	Package crc32 implements the 32-bit cyclic redundancy check, or CRC-32, checksum.	Used to compute checksum of packets.
<i>os.exec</i>	Package exec runs external commands. It wraps os.StartProcess to make it easier to remap stdin and stdout, connect I/O with pipes, and do other adjustments. https://pkg.go.dev/os	Used to execute operating system commands.
<i>path/filepath</i>	Package filepath implements utility routines for manipulating filename paths in a way compatible with the target operating system-defined file paths. https://pkg.go.dev/path/filepath	Used for file parsing.

The next section is the reflection on the incident.

Reflection

The discovery, containment, and eradication of the covert malware on msi-pro workstation became a crucible in which our blue-team capabilities were tested end-to-end. The episode compressed every phase of the incident-response lifecycle—from detecting suspicious network traffic to the final lessons-learned—into nine intense days that illuminated resilience.

We checked for host and network anomalies and were able to identify the port knock used to activate the malware on msi-pro machine, and also the inbound and outbound traffic to and from the command and control server. We blocked all traffic to and from the command and control server, and wrote Snort rules to detect the port knock and block the machine performing the port knock. We killed the malicious process and cleared all cron jobs to prevent the malware from persisting on the msi-pro machine.

Moreover, we captured network traffic pcaps and also tried reverse engineer the malware to understand its behavior. The malware used /tmp as its working directory, and made minimal changes to the msi-pro workstation. Therefore, tools such as AIDE and Auditd failed to detect abnormal file system changes. Also, the malware was new, so signature based detection failed. In addition, we reviewed what went wrong, and what we could have done better. For instance, we should have been updating all packages on the system regularly. And we should have set up AppArmor beforehand. And also, we should have been reviewing the cron jobs and process tree regularly.